DevOps Question Bank Linux, Shell Scripting & Networking

86 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Linux, Shell Scripting & Networking	22	21	43	86

Questions in this topic

Basic 6

Q1. How do you create a new user in Linux, and where is the default home directory created? 6

Q2. How do you search for files containing a specific string, and print only the file names? 6

Q3. How would you validate command-line arguments in a shell script? 7

Q4. How do Linux file permissions work (chmod, chown)? 8

Q5. How do you manage services and read their logs (systemd, systemctl, journalctl)? 9

Q6. What is cron and how do you schedule a job? 9

Q7. Which commands do you use to check CPU, memory and disk on a Linux server? 10

Q8. How do you check running processes in Linux? 11

Q9. What command finds files larger than 100MB? 11

Q10. Which commands do you use for basic network troubleshooting, and how do you check connectivity between two servers? 12

Q11. How much Linux do you use in DevOps work, and which distributions have you worked with? . . . 13

Q12. How does DNS work? 14

Q13. Explain TCP/IP basics: TCP vs UDP, ports and the three-way handshake. 15

Q14. How does TLS/HTTPS work? 15

Q15. What is the difference between a load balancer and a reverse proxy? 16

Q16. How do you find files modified in the last 10 minutes? 17

Q17. What is Linux patching, why does it matter, and what types of patches are there? 18

Q18. How do you check a package version, check available updates, and update all packages? 19

Q19. How do you find the OS version on Linux? 20

Q20. How do you search for a specific file inside subfolders? 20

Q21. How do you check which ports are open or listening on a Linux server? 21

Q22. What is the difference between curl and wget? 21

Advanced 22

Q23. Write a shell script to verify whether SSH connectivity to a server (or list of servers) is working. . . 22

Q24. How can you access a Windows C: drive from an Ubuntu system (dual boot or mounted disk)? . . 23

Q25. How do you troubleshoot network latency step by step? 24

Q26. How do you approach troubleshooting and debugging automation (shell) scripts? 25

Q27. Can you share an example of a complex automation script you have written? 25

Q28. Write a shell script to find prime numbers (e.g. all primes up to N). 26

Q29. How do you extend a partition or filesystem without unmounting it? 27

Q30. What are the steps to add a new disk to a Linux server? 28

Q31. How do you create and mount a swap file? . . . 29
Q32. How do you find which processes are writing to a file in real time? . . . 30
Q33. What is the fastest way to copy a huge file (or many files) across servers? . . . 30
Q34. How do you analyse disk I/O performance on Linux? . . . 31
Q35. How do you roll back a bad patch or package update? . . . 32
Q36. How do you apply a kernel patch, reboot safely, and keep the old kernel available? . . . 33
Q37. What is live kernel patching? . . . 34
Q38. How do you patch a fleet of servers in production? . . . 34
Q39. In Nginx, how do you make a client stick to a specific server (sticky sessions)? . . . 35
Q40. In Nginx, with sticky sessions or ip_hash, what happens if a server goes down? . . . 36
Q41. How does header management work in Nginx? . . . 37
Q42. How do you identify and manage parent-child process relationships in Linux? . . . 38
Q43. Write a script to convert human-readable file sizes (e.g. 1.5K, 20M, 3G) to bytes. . . . 39

Scenario-Based **40**

Q44. An Ubuntu OS upgrade failed midway. How do you troubleshoot and recover? . . . 40
Q45. A Linux node shows high CPU usage even though all the pods on it appear healthy. What could be the reasons? . . . 41
Q46. Disk utilization on production servers reached 95%. What immediate and long-term actions would you take? . . . 42
Q47. One of your EC2/Linux nodes keeps going out of memory. How would you diagnose and prevent it? 43
Q48. Latency spikes suddenly in production during off-peak hours. Where do you start debugging? . . . 45
Q49. A user cannot log in to a Linux server. How do you troubleshoot? . . . 45
Q50. A script is executable by one user but not another, or a user cannot access a directory even though they are in the correct group. Why? . . . 46
Q51. You cannot SSH into a server. How do you troubleshoot? . . . 47
Q52. Ping to a server works, but SSH using its hostname fails. Why? . . . 48
Q53. A log file shows junk characters when you open it. What could be wrong? . . . 48
Q54. A cron job is not running. How do you debug it? . . . 49
Q55. yum/dnf or apt is failing. How do you troubleshoot it? . . . 50
Q56. A user is in the sudoers file but sudo still fails. Why? . . . 50
Q57. A server hit a kernel panic. How do you troubleshoot and recover? . . . 51
Q58. How do you find and handle zombie processes? . . . 52
Q59. An application fails with "Too many open files". How do you fix it? . . . 52
Q60. A user's home directory is missing. How do you restore it? . . . 53
Q61. The server time is wrong. How do you fix it using NTP/chrony? . . . 54
Q62. You deleted large files but disk space was not freed. Why? What happens when a file is deleted while still in use? . . . 54

Q63. df and du show different disk usage. Why? . . . 55
Q64. The system load is very high. How do you analyse it? . . . 56
Q65. An NFS mount is not working. How do you troubleshoot it? . . . 56
Q66. How do you recover a deleted file on Linux? . . . 57
Q67. A filesystem went read-only. How do you remount it read-write without a reboot? . . . 58
Q68. How do you fix a corrupted filesystem with fsck? . . . 59
Q69. The disk shows free space but you get "No space left on device". Why? (inode exhaustion) . . . 59
Q70. Apache is running, but the website is not accessible. How do you troubleshoot it? . . . 60
Q71. Automate an application deployment with a shell script, with backup, health check and automatic rollback. . . . 61
Q72. An application fails intermittently. How do you use Linux command-line tools to analyse its logs? . 62
Q73. A critical service fails to start after a reboot. How do you troubleshoot it? . . . 63
Q74. How would you tune the kernel with sysctl for a high-traffic application? . . . 64
Q75. How do you deploy updates with zero downtime on a plain Linux server (no Kubernetes)? . . . 65
Q76. Design a 3-tier architecture (web, app, DB), explain the request flow, and debug a failure layer by layer with Linux commands. . . . 66
Q77. You are locked out of a server over SSH and have no root access. How do you recover? . . . 67
Q78. Write a shell script that checks whether a service is running, restarts it if not, and logs the event. . 68
Q79. Networking: a remote user cannot connect to the company VPN. How do you troubleshoot? . . . 69
Q80. Networking: users report slow access between two branch offices over a WAN link (or a remote office cannot reach the main server). How do you troubleshoot? . . . 70
Q81. Networking: the network has intermittent connectivity issues. What could cause it, and how do you troubleshoot? . . . 71
Q82. Networking: traffic is not being forwarded between two subnets. How do you troubleshoot the routing?
Q83. Networking: you need to add a new subnet to an existing network. What are the steps? . . . 72
Q84. Networking: you notice abnormal traffic patterns on an interface. How do you investigate and what do you recommend? . . . 73
Q85. Networking: how do you implement redundancy for critical network devices and links? . . . 74
Q86. Networking: a router is not learning routes from its neighbours (BGP/OSPF). How do you troubleshoot? . . . 75

Linux, Shell Scripting & Networking

Basic

Q1. How do you create a new user in Linux, and where is the default home directory created?

> **In simple words:** Creating a user is like giving a new employee an ID card plus a personal locker. The ID card is the account, the locker is the home folder where their personal stuff lives.

Detailed answer

Use `useradd` (low-level) or `adduser` (friendly, interactive wrapper on Debian/Ubuntu).

- `useradd -m -s /bin/bash priya` -> `-m` creates the home directory, `-s` sets the login shell.
- Default home location is **/home/<username>** (for root it is `/root`). The default comes from `/etc/default/useradd` (`HOME=/home`) and `/etc/login.defs`.
- New home folders are pre-filled with files copied from **/etc/skel** (like `.bashrc`).
- Set password with `passwd priya`; add to a group like sudo with `usermod -aG sudo priya` (on RHEL/Amazon Linux the admin group is `wheel`).
- User details are stored in `/etc/passwd`, hashed passwords in `/etc/shadow`.

Note: plain `useradd` on some distros does NOT create the home folder unless you pass `-m`.

Example:

```
sudo useradd -m -s /bin/bash -c "Priya DevOps" priya
sudo passwd priya
sudo usermod -aG sudo priya
grep priya /etc/passwd
# priya:x:1001:1001:Priya DevOps:/home/priya:/bin/bash
ls -la /home/priya          # .bashrc .profile copied from /etc/skel
```

> ***What to say to the interviewer:*** *I use* `useradd -m -s /bin/bash <user>` *or* `adduser` *on Ubuntu, then* `passwd` *and* `usermod -aG` *for groups. The home directory is created under /home/<user> by default, seeded from /etc/skel, and the defaults come from /etc/default/useradd and /etc/login.defs.*

Q2. How do you search for files containing a specific string, and print only the file names?

(also asked as: find all files containing a specific sentence and print only the file names)

> **In simple words:** It is like asking a librarian "which books mention the word X?" and wanting just the list of book titles, not every page where it appears.

Detailed answer

`grep` is the tool. Key flags:

- `-r` / `-R` -> search recursively through folders.

- `-l` -> print only the **file names** that match (not the matching lines).
- `-i` -> ignore case; `-n` -> show line numbers; `-w` -> whole word only.
- `-F` -> treat the pattern as a fixed string (safe for sentences with dots or brackets).
- `--include="*.log"` -> limit to certain file types.
- For a full sentence, wrap it in quotes so the shell passes it as one argument.

You can also combine `find` + `grep` when you need filters like age or size.

Example:

```
# File names only, recursive, exact sentence
grep -rlF "Connection refused by upstream" /var/log/

# Case-insensitive, only .conf files
grep -rli --include="*.conf" "max_connections" /etc/

# With find: only files modified in last 2 days
find /var/log -type f -mtime -2 -exec grep -lF "OutOfMemory" {} +

# Count of matching files
grep -rlF "ERROR" /app/logs | wc -l
```

> ***What to say to the interviewer:*** *I use* `grep -rl "text" /path` `-r` *for recursive and* `-l` *to print only file names; for a full sentence I add* `-F` *so it is matched literally. If I need extra filters like file age or type, I combine it with* `find ... -exec grep -l`.

Q3. How would you validate command-line arguments in a shell script?

> **In simple words:** Before a chef starts cooking, they check the ingredients are actually on the counter. A script should also check it was given the right inputs before doing anything risky.

Detailed answer

Useful building blocks:

- `$#` -> number of arguments; `$1`, `$2` -> individual arguments; `$@` -> all arguments.
- Check count: `if [ $# -ne 2 ]; then usage; fi`.
- Check empty: `[ -z "$1" ]`; check file exists: `[ -f "$file" ]`; directory: `[ -d "$dir" ]`.
- Check a number with a regex: `[[ "$port" =~ ^[0-9]+$ ]]`.
- Use `getopts` for flags like `-e prod -v`.
- Print a `usage` message and `exit 1` (non-zero = failure) so CI pipelines notice the error.
- Add `set -euo pipefail` at the top so the script stops on errors and unset variables.

Example:

```
#!/usr/bin/env bash
set -euo pipefail

usage() { echo "Usage: $0 <env: dev|stage|prod> <port>"; exit 1; }

[ $# -eq 2 ] || usage
ENV="$1"; PORT="$2"

case "$ENV" in
  dev|stage|prod) ;;
  *) echo "Invalid env: $ENV"; usage ;;
esac

[[ "$PORT" =~ ^[0-9]+$ ]] && [ "$PORT" -le 65535 ] || { echo "Invalid port"; exit 2; }

echo "Deploying to $ENV on port $PORT"
```

What to say to the interviewer: *I check* `$#` *for the argument count, validate each value with tests like* `-z`, `-f` *or a regex, and use* `getopts` *for flags. On bad input I print a usage message and exit with a non-zero code, and I start scripts with* `set -euo pipefail` *so they fail fast.*

Q4. How do Linux file permissions work (chmod, chown)?

In simple words: Every file has a lock with three keyholders: the owner, the owner's team (group), and everyone else. Each can be allowed to read, write, or run it.

Detailed answer

`ls -l` shows something like `-rwxr-x---`:

- 1st char: type (`-` file, `d` directory, `l` link).
- Next 3 groups: **user / group / others**, each with `r` (read=4), `w` (write=2), `x` (execute=1).
- `chmod 750 deploy.sh` -> owner rwx (7), group r-x (5), others nothing (0).
- `chmod +x script.sh` -> make it executable.
- `chown appuser:appgroup file` -> change owner and group.
- On directories, `x` means "can enter the folder".
- Special bits: setuid, setgid, sticky bit (`/tmp` uses sticky so users cannot delete each other's files).
- Private keys must be `600` or SSH will refuse them.

Example:

```
ls -l deploy.sh
# -rw-r--r-- 1 ubuntu ubuntu 512 Sep 20 10:00 deploy.sh
chmod 750 deploy.sh
sudo chown jenkins:devops deploy.sh
chmod 600 ~/.ssh/id_rsa
ls -ld /tmp     # drwxrwxrwt  <- 't' = sticky bit
```

What to say to the interviewer: *Permissions are read/write/execute for user, group and others, shown by* `ls -l` *and set with* `chmod` *either symbolically or in octal like 755 or 600. I use* `chown` *to change ownership, and I follow least privilege, for example SSH keys at 600 and scripts at 750.*

Q5. How do you manage services and read their logs (systemd, systemctl, journalctl)?

In simple words: systemd is the building manager that starts all the machines (services) when the building opens and restarts them if they break. journalctl is the manager's logbook.

Detailed answer

- `systemctl status nginx` -> is it running, recent log lines, PID.
- `systemctl start|stop|restart|reload nginx`.
- `systemctl enable nginx` -> start automatically on boot.
- `systemctl list-units --failed` -> what is broken right now.
- `journalctl -u nginx -f` -> follow logs for one service.
- `journalctl -u nginx --since "1 hour ago"`; `journalctl -p err -b` -> errors since last boot.
- `journalctl --disk-usage` and `journalctl --vacuum-size=500M` -> control log size.
- Unit files live in `/etc/systemd/system/`; after editing run `systemctl daemon-reload`.
- `Restart=on-failure` in a unit file gives auto-healing.

Example:

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My App
After=network.target

[Service]
User=appuser
ExecStart=/opt/myapp/bin/start.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now myapp
journalctl -u myapp -f
```

What to say to the interviewer: *I use* `systemctl` *to start, stop, enable and check services, and* `journalctl -u <service>` *to read their logs with filters like* `--since` *and* `-p err`*. For custom apps I write a unit file with* `Restart=on-failure` *so systemd auto-restarts it.*

Q6. What is cron and how do you schedule a job?

In simple words: cron is an alarm clock for the server. You tell it "at 2 AM every night, run this script" and it does it without anyone logging in.

Detailed answer

- Edit with `crontab -e` (per user) or put files in `/etc/cron.d/`.
- Format: `minute hour day-of-month month day-of-week command`.

- `*/5 * * * *` -> every 5 minutes; `0 2 * * *` -> 2:00 AM daily; `0 9 * * 1-5` -> 9 AM weekdays.
- cron runs with a **minimal PATH/environment** -> use full paths inside scripts.
- Always redirect output to a log: `>> /var/log/job.log 2>&1`, otherwise failures are silent.
- Check it ran: `grep CRON /var/log/syslog` (Ubuntu) or `journalctl -u cron`.
- Use `flock` to avoid two copies running at the same time.
- Modern alternatives: systemd timers, Kubernetes CronJobs.

Example:

```
crontab -e
# Clean logs older than 7 days every night at 2 AM
0 2 * * * /usr/bin/find /var/log/myapp -name "*.log" -mtime +7 -delete >> /var/log/cleanup.log 2>&1
# Health check every 5 min, never overlapping
*/5 * * * * /usr/bin/flock -n /tmp/hc.lock /opt/scripts/healthcheck.sh >> /var/log/hc.log 2>&1

crontab -l      # list jobs
```

> ***What to say to the interviewer:*** *cron schedules recurring jobs using five time fields, edited via* `crontab -e`*. I always use absolute paths, redirect output to a log file, and use* `flock` *to prevent overlapping runs; in Kubernetes the equivalent is a CronJob.*

Q7. Which commands do you use to check CPU, memory and disk on a Linux server?

> **In simple words:** These are the dashboard gauges of a car: speed (CPU), fuel (memory), and boot space (disk).

Detailed answer

Need	Command	What to look at
CPU/processes	`top`, `htop`	%CPU, load average, `wa` (IO wait), `st` (steal)
Load	`uptime`	load average vs number of cores (`nproc`)
Memory	`free -h`	the **available** column, not "free"
Disk space	`df -h`	Use% per filesystem
Inodes	`df -i`	many tiny files can fill inodes
Folder sizes	`du -sh /var/*` then sort -h	largest directories
Disk IO	`iostat -x 1`, `iotop`	%util, await
Per-process history	`vmstat 1`, `sar`	trends over time

Linux uses spare RAM as file cache, so low "free" memory is normal; watch "available" and swap usage.

Example:

```
nproc; uptime
free -h
df -h
du -xh / --max-depth=1 2>/dev/null | sort -h | tail
ps aux --sort=-%cpu | head -5
ps aux --sort=-%mem | head -5
```

What to say to the interviewer: *I start with* `top`/`htop` *and* `uptime` *for CPU and load,* `free -h` *for memory looking at the available column, and* `df -h` *plus* `du -sh` *for disk. For deeper issues I use* `iostat`, `vmstat` *and* `ps --sort` *to find the heaviest processes.*

Q8. How do you check running processes in Linux?

In simple words: Processes are the workers currently busy in the office. `ps` is a snapshot of the attendance sheet, `top` is a live CCTV view, and `pgrep` is calling out one worker by name.

Detailed answer

- `ps aux` -> snapshot of all processes (user, PID, %CPU, %MEM, command).
- `ps -ef --forest` -> shows parent/child tree (who started whom).
- `top` / `htop` -> live, sortable view (press `P` for CPU, `M` for memory in top).
- `pgrep -a nginx` -> find PIDs by name; `pidof nginx` similar.
- `systemctl status nginx` -> service state plus its main PID and child processes.
- `ps -o pid,ppid,stat,etime,cmd -p <PID>` -> details like uptime (`etime`) and state (`R` running, `S` sleeping, `D` stuck on IO, `Z` zombie).
- `lsof -p <PID>` -> files and sockets the process has open; `ss -tnp` -> its network connections.
- Stop it: `kill <PID>` (graceful SIGTERM), `kill -9 <PID>` only as last resort (SIGKILL, no cleanup).

Example:

```
ps aux --sort=-%cpu | head -5
ps -ef --forest | grep -A3 nginx
pgrep -a java
# 2143 java -Xmx2g -jar /opt/app/app.jar
ps -o pid,stat,etime,%mem,cmd -p 2143
ps aux | awk '$8 ~ /Z/'        # zombie processes
```

What to say to the interviewer: *I use* `ps aux` *or* `ps -ef --forest` *for a snapshot,* `top`/`htop` *for live usage, and* `pgrep` *or* `systemctl status` *to find a specific process. For deeper checks I look at process state and uptime with* `ps -o`*, and open files or sockets with* `lsof` *and* `ss -p`*.*

Q9. What command finds files larger than 100MB?

(also asked as: How do you find large unused files across partitions? / Which directory is too big?)

In simple words: Like walking through a warehouse with a scale and putting a sticker on every box heavier than 100 kg, so you know exactly what is taking up the space.

Detailed answer

- `find / -xdev -type f -size +100M` -> all files over 100MB; `-xdev` stays on one filesystem (skips /proc and mounted volumes).
- Add `-exec ls -lh {} +` or `-printf` to show sizes, and `sort` to rank them.
- `du -ah /var | sort -rh | head -20` -> largest files AND folders under a path.
- `ncdu /` -> interactive, easy browsing of disk usage.
- Combine with age: `-mtime +30` for big files not modified in 30 days (safe cleanup candidates).
- "Unused" = not read recently: `-atime +90` (last access). Many servers mount with `relatime`, so atime is only roughly accurate - treat it as a hint, not proof.
- To scan every partition, run `find` once per mount point from `df -h` output, each with `-xdev`, so network mounts (NFS) are not crawled by accident.
- Redirect errors with `2>/dev/null` to hide "Permission denied" noise.

Example:

```
sudo find / -xdev -type f -size +100M -printf '%s\t%p\n' 2>/dev/null \
  | sort -rn | head -10 | awk '{printf "%.1f MB\t%s\n", $1/1048576, $2}'
# 2310.4 MB  /var/log/myapp/app.log
# 845.2 MB   /opt/exports/dump-2026-08.sql

sudo du -ah /var 2>/dev/null | sort -rh | head -20
find /opt/exports -type f -size +100M -mtime +30 -ls
```

What to say to the interviewer: *I use* `find / -xdev -type f -size +100M`*, usually with* `-printf` *and* `sort` *to rank by size, and* `du -ah | sort -rh | head` *to see the biggest folders too. Adding* `-mtime +30` *gives me old, large files that are the safest candidates for cleanup.*

Q10. Which commands do you use for basic network troubleshooting, and how do you check connectivity between two servers?

(also asked as: How do you check network connectivity between two servers? / A server cannot reach a specific external IP - how do you troubleshoot?)

In simple words: Think of sending a parcel. `dig` checks the address book, `ping` checks the road exists, `traceroute` shows each town on the way, `ss` shows which doors in the building are open, and `curl` actually knocks and waits for a reply.

Detailed answer

- `ss -tulnp` (or older `netstat -tulnp`) -> which ports are listening and which process owns them.
- `dig api.example.com` / `nslookup` -> DNS resolution.
- `ping host` -> basic reachability (ICMP may be blocked in cloud - not proof of failure).
- `traceroute` / `mtr` -> where along the path packets are lost or slow.
- `curl -v https://host/health` -> full HTTP/TLS conversation.
- `nc -zv host 5432` or `telnet host 5432` -> is a TCP port reachable.
- `ip a`, `ip route` -> interfaces and routes.

- `tcpdump -i eth0 port 443` -> capture packets for deep debugging.

Server A -> Server B checklist: resolve the name (`dig`), test the exact port from A (`nc -zv B 5432`), confirm B is listening on the right interface (`ss -tlnp` on B - `127.0.0.1` means local only, `0.0.0.0` means all), then check the firewalls in between: OS firewall (`ufw status`, `iptables -L`), cloud **Security Groups** (inbound on B, outbound on A), **NACLs** (stateless - need return ephemeral ports too), and route tables/peering. Timeout = blocked somewhere; refused = reached B but nothing listening.

Cannot reach one specific external IP (others work): test the IP directly to rule out DNS (`curl -v https://<ip>` vs the name), check the route it takes (`ip route get <ip>` - a VPN or peering route may send it the wrong way), `traceroute`/`mtr` to see where it stops, the local firewall (`iptables -L -n`, `nft list ruleset`), outbound Security Group/NACL rules and the NAT gateway for private subnets, whether the remote side allow-lists your source IP (your NAT/Elastic IP), and **MTU** issues (small pings work but big packets hang: `ping -M do -s 1472 <ip>`).

Example:

```
ss -tulnp | grep 8080
dig +short db.internal.example.com
nc -zv 10.0.2.15 5432
# Connection to 10.0.2.15 5432 port [tcp/postgresql] succeeded!
curl -sv -o /dev/null https://api.example.com/health
mtr -rwc 20 api.example.com
```

> ***What to say to the interviewer:*** *I go layer by layer -* `dig` *for DNS,* `ping`*/*`mtr` *for reachability,* `nc -zv` *to test the TCP port,* `ss -tulnp` *to confirm the service is listening, and* `curl -v` *to see the HTTP and TLS response. If still unclear I capture traffic with* `tcpdump`*.*

Q11. How much Linux do you use in DevOps work, and which distributions have you worked with?

> **In simple words:** Linux is the ground almost every DevOps building stands on - servers, containers and Kubernetes nodes all run on it, so a DevOps engineer lives in the Linux terminal daily.

Detailed answer

Speak from real usage:

- **Distros:** Ubuntu (22.04/24.04 LTS) for servers like monitoring boxes, Amazon Linux 2023 on AWS EC2/EKS nodes, sometimes RHEL/CentOS/Rocky, and Alpine or Debian-slim as container base images.
- **Daily tasks:** SSH/SSM into servers, reading logs (`journalctl`, `tail -f`, `grep`), checking resources (`top`, `df`, `free`), managing services with `systemctl`, package management (`apt`, `dnf/yum`), cron jobs, file permissions, writing Bash automation, networking checks (`curl`, `dig`, `ss`).
- **Examples you can mention:** cleaning a nearly full disk on a monitoring server with `journalctl --vacuum`, tuning a service to fix memory pressure, writing health-check scripts.

Key differences worth knowing: Debian family uses `apt` and `.deb`; RHEL family uses `dnf`/`yum` and `.rpm`.

Example:

```
cat /etc/os-release
# PRETTY_NAME="Ubuntu 22.04.4 LTS"
uname -r          # kernel version
apt update && apt list --upgradable   # Ubuntu
dnf check-update                      # Amazon Linux 2023 / RHEL
```

> ***What to say to the interviewer:*** *I use Linux every day - mainly Ubuntu LTS and Amazon Linux on AWS, plus Alpine and Debian-slim in containers. My daily work includes log analysis, service management with systemd, resource troubleshooting, package management, cron and Bash automation.*

Q12. How does DNS work?

Also asked as: What is DNS caching?

> **In simple words:** DNS is the phone book of the internet. You know the name "google.com", DNS looks up the phone number (IP address) so your computer can call it.

Detailed answer

Flow when you type `api.example.com`:

1. Check the local cache (browser, OS, `/etc/hosts`).
2. Ask the configured **recursive resolver** (from `/etc/resolv.conf`, e.g. VPC resolver, 8.8.8.8).
3. Resolver asks a **root server** -> "ask the .com servers".
4. .com **TLD server** -> "ask example.com's authoritative name servers".
5. **Authoritative server** (e.g. Route 53) returns the record.
6. Answer is cached for the record's **TTL**.

Common record types: `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias to another name), `MX` (mail), `TXT` (verification), `NS` (name servers). Low TTL = faster changes but more queries. Many "network outages" are actually DNS issues - stale cache, wrong record, or resolver overload.

Example:

```
dig api.example.com +short
dig api.example.com +trace          # see root -> TLD -> authoritative
dig @8.8.8.8 api.example.com        # ask a specific resolver
cat /etc/resolv.conf
```

DNS caching: every layer keeps answers for the record's **TTL**, so repeat lookups are fast and don't hit the authoritative servers again.

- **Layers:** the browser or application (the JVM caches DNS, sometimes forever, so set `networkaddress.cache.ttl`), the OS stub resolver (`systemd-resolved`, `nscd`), the recursive resolver (ISP, 8.8.8.8, the AWS VPC resolver), and in Kubernetes CoreDNS's cache plus NodeLocal DNSCache.
- **Negative caching:** a failed lookup (NXDOMAIN) is also cached, so a record you just created may seem "missing" for a while.
- **Why it matters in DevOps:** lower the TTL (for example to 60s) a day **before** a migration or failover so clients pick up the new IP quickly, then raise it again afterwards.

```
resolvectl statistics           # cache hits and misses (systemd-resolved)
sudo resolvectl flush-caches    # flush the local cache
dig +noall +answer example.com  # the TTL is the second column and counts down on repeat queries
```

> ***What to say to the interviewer:*** *DNS translates names into IP addresses: the client asks a recursive resolver, which walks from root to TLD to the authoritative server and caches the answer for the TTL. I debug it with* `dig`*, including* `+trace` *and querying specific resolvers.*

Q13. Explain TCP/IP basics: TCP vs UDP, ports and the three-way handshake.

> **In simple words:** TCP is a registered courier - it confirms delivery and re-sends lost parcels. UDP is dropping flyers in letterboxes - faster, but no guarantee each one arrives.

Detailed answer

- **IP** gives each machine an address and routes packets between networks.
- **Ports** identify the application on that machine (22 SSH, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 3306 MySQL, 6379 Redis, 53 DNS).
- **TCP**: connection-based, ordered, reliable (retransmits). Used by HTTP, SSH, databases.
- **UDP**: connectionless, no guarantee, low overhead. Used by DNS, video streaming, VoIP, NTP.
- **Three-way handshake**: client sends `SYN` -> server replies `SYN-ACK` -> client sends `ACK` -> connection open. Closing uses `FIN`/`ACK`.
- Every handshake costs one round trip, which is why connection reuse (keep-alive, pooling) reduces latency.
- `Connection refused` = host reachable but nothing listening; `timeout` = packet dropped (firewall/security group/routing).

Example:

```
sudo tcpdump -i eth0 -nn port 443 -c 3
# 10.0.1.5.50412 > 52.1.2.3.443: Flags [S]    <- SYN
# 52.1.2.3.443 > 10.0.1.5.50412: Flags [S.]   <- SYN-ACK
# 10.0.1.5.50412 > 52.1.2.3.443: Flags [.]    <- ACK
```

> ***What to say to the interviewer:*** *TCP is reliable and connection-oriented with a SYN, SYN-ACK, ACK handshake, while UDP is faster but best-effort. A "connection refused" usually means nothing is listening on the port, while a timeout usually points to a firewall, security group or routing problem.*

Q14. How does TLS/HTTPS work?

> **In simple words:** TLS is like sending a letter in a locked box. The website first shows an ID card (certificate) signed by a trusted authority, then you both agree on a secret key so nobody in between can read the letter.

Detailed answer

1. Client connects and says which TLS versions/ciphers it supports and the hostname (SNI).
2. Server sends its **certificate** (public key + domain name, signed by a Certificate Authority).
3. Client verifies: trusted CA chain, not expired, domain matches.

4. Both sides use key exchange (e.g. ECDHE) to create a shared **session key**.
5. All data is encrypted with that fast symmetric key.

DevOps-relevant points:

- Certificates expire -> automate renewal (Let's Encrypt/cert-manager, AWS ACM) and alert before expiry.
- **TLS termination** often happens at the load balancer/Ingress.
- Common errors: expired cert, missing intermediate chain, hostname mismatch, old TLS version.
- **mTLS** = both sides show certificates (used in service meshes).

Example:

```
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null \
  | openssl x509 -noout -subject -issuer -dates
# notAfter=Dec 30 23:59:59 2026 GMT
curl -vI https://api.example.com 2>&1 | grep -E "SSL|expire|subject"
```

> ***What to say to the interviewer:*** *In TLS the server presents a CA-signed certificate, the client validates it, and both negotiate a symmetric session key so traffic is encrypted. Operationally I automate certificate renewal, monitor expiry, and debug with* `openssl s_client` *and* `curl -v`*.*

Q15. What is the difference between a load balancer and a reverse proxy?

(also asked as: How would you load balance 3-4 servers, which type is preferred and why?)

> **In simple words:** A reverse proxy is a receptionist at the front desk. Visitors never walk straight into the offices; the receptionist checks IDs, answers common questions and passes messages on, even when there is only one office. A load balancer is a traffic warden who spreads visitors across several identical counters so no one counter gets overloaded.

Detailed answer

- **Reverse proxy**: sits in front of one or more servers and talks to clients on their behalf. Jobs: **TLS termination**, caching, compression, header rewrites, path-based routing (`/api` to one app, `/` to another), authentication, rate limiting and WAF. It is useful even with a **single** backend.
- **Load balancer**: spreads traffic across **multiple** backends, with **health checks** (unhealthy servers are removed) and algorithms such as round robin, least connections and IP/hash-based stickiness. It works at **L4** (TCP/UDP: sees IPs and ports) or **L7** (HTTP: sees URLs, headers, cookies).
- They overlap a lot: nginx, HAProxy and Envoy do both. An **AWS ALB** is an L7 load balancer that also acts as a reverse proxy (TLS, path/host rules). An **AWS NLB** is an L4 load balancer that forwards connections and is not an HTTP proxy.

Aspect	Reverse proxy	Load balancer
Main goal	shield and enhance backends	distribute load, high availability
Needs many backends?	no, one is fine	yes, that is the point
OSI layer	usually L7 (HTTP)	L4 or L7

Aspect	Reverse proxy	Load balancer
Typical features	TLS, cache, compression, rewrites, auth, WAF	health checks, algorithms, stickiness, failover
Examples	nginx, Apache mod_proxy, Envoy, Traefik	AWS ALB/NLB, HAProxy, nginx upstream, F5

Load balancing 3-4 servers - which type and why?

- Algorithms: **round robin** (default, each server in turn), **weighted** round robin (`weight=3` for a bigger server), **least connections** (`least_conn`, send to the least busy), **ip_hash** / **hash** (same client or key -> same server, for stickiness or cache locality), and **random two choices** (`random two least_conn`: pick 2 at random, use the less loaded one - good with many LB instances).
- **L4 vs L7**: L4 (TCP, e.g. NLB, HAProxy `mode tcp`) is faster and protocol-agnostic - use it for databases, MQTT, gRPC passthrough or huge throughput. L7 (HTTP, e.g. ALB, nginx) can route by path/host, terminate TLS, retry and see HTTP errors.
- **Health checks** are what make it a load balancer and not just a splitter: remove failed servers automatically (passive `max_fails` in open-source nginx, active checks in HAProxy/ALB).
- Preferred for HTTP APIs: **L7 with least connections**, because request durations vary - round robin can pile slow requests onto one server, while least_conn follows the real load, and L7 gives health checks on real HTTP status plus TLS and routing in one place. Keep the app stateless so no stickiness is needed.

Example:

```
upstream app_backend {
    least_conn;                                 # load balancing algorithm
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
}
server {
    listen 443 ssl;                             # reverse proxy: TLS termination
    server_name app.example.com;
    ssl_certificate     /etc/nginx/tls/app.crt;
    ssl_certificate_key /etc/nginx/tls/app.key;
    gzip on;
    location /api/ {
        proxy_pass http://app_backend;          # load balanced across 2 servers
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

What to say to the interviewer: *A reverse proxy sits in front of servers and handles TLS termination, caching, compression, routing and security, even with a single backend. A load balancer spreads traffic across many backends with health checks and algorithms at L4 or L7. In practice nginx, HAProxy and an AWS ALB do both, while an NLB is a pure L4 load balancer.*

Q16. How do you find files modified in the last 10 minutes?

In simple words: Like asking "who touched anything in this room in the last 10 minutes?" - useful right after an incident or a deployment to see exactly what changed.

Detailed answer

- `find /path -type f -mmin -10` -> modified in the last 10 minutes (`-mmin` = minutes, `-mtime` = days).
- `-mmin +10` means MORE than 10 minutes ago; `-10` means LESS than 10 minutes ago.
- `-cmin -10` -> metadata changed (permissions/owner) in the last 10 minutes.
- `-amin -10` -> accessed recently (depends on `relatime` mount option).
- `-newer reference.file` -> changed after another file, handy with a marker made by `touch` before a deploy.
- Add `-xdev` and `2>/dev/null` when scanning `/`, and exclude `/proc` and `/sys`.

Example:

```
sudo find /etc /opt/app -type f -mmin -10 -ls
sudo find / -xdev -type f -mmin -10 2>/dev/null | grep -v -E "^/(proc|sys)"
touch /tmp/before_deploy; ./deploy.sh
find /opt/app -newer /tmp/before_deploy -type f
```

> ***What to say to the interviewer:*** *I use* `find /path -type f -mmin -10`*, where the minus sign means "less than 10 minutes ago", and* `-cmin` *if I also care about permission or ownership changes. For deployments I touch a marker file first and use* `find -newer` *to list exactly what changed.*

Q17. What is Linux patching, why does it matter, and what types of patches are there?

(also asked as: What are the ways to apply patches? / Security patch vs kernel patch?)

> **In simple words:** Patching is like the regular service of a car - fixing known faults (bugs) and fitting new locks (security fixes) before a thief finds the weak one.

Detailed answer

Patching = updating OS packages to newer, fixed versions. It matters because most breaches exploit **known, already-fixed** vulnerabilities (CVEs); it is also needed for compliance (PCI, ISO 27001, SOC 2) and stability.

Type	What it fixes	Reboot?
Security patch	a CVE in a package (openssl, openssh, sudo)	usually no - restart the service
Bug-fix / feature update	normal package updates	usually no
Kernel patch	the Linux kernel itself	**yes** (or live patch)

Ways to apply: package manager by hand (`apt`, `dnf/yum`), unattended/automatic updates (`unattended-upgrades`, `dnf-automatic`) for security-only, config management at scale (Ansible), cloud tools (**AWS SSM Patch Manager**), live patching for kernels, or immutable infra - bake a new patched AMI/image and replace servers.

Example:

```
sudo apt list --upgradable 2>/dev/null | grep -i security    # Ubuntu
sudo dnf updateinfo list security                            # RHEL / Amazon Linux
sudo dnf upgrade --security -y
```

What to say to the interviewer: *Patching means updating packages to close known vulnerabilities and bugs, which matters because most attacks use CVEs that already have fixes. Security and normal package patches usually only need a service restart, while kernel patches need a reboot or live patching, and at scale I apply them with Ansible or SSM Patch Manager, or by rebaking images.*

Q18. How do you check a package version, check available updates, and update all packages?

In simple words: Before servicing the car you read the current part numbers, check which new parts are available, then fit them - and write down the old numbers in case you need to go back.

Detailed answer

Task	Debian/Ubuntu (apt)	RHEL/Amazon Linux (dnf/yum)
Installed version	`dpkg -l openssl` or `apt policy openssl`	`rpm -q openssl` or `dnf info openssl`
Refresh repo metadata	`apt update`	`dnf makecache`
List available updates	`apt list --upgradable`	`dnf check-update` (or `yum check-update`)
Update one package	`apt install --only-upgrade openssl`	`dnf update openssl`
Update everything	`apt upgrade` (or `full-upgrade`)	`dnf upgrade` (or `yum update`)
History	`/var/log/apt/history.log`	`dnf history`

Note: it is `dpkg -l` (lowercase L for "list"), not `dpkg -1`. `apt upgrade` never removes packages; `full-upgrade` may add or remove packages to resolve dependencies. Always note current versions before patching so a rollback is possible.

Example:

```
dpkg -l | grep -E "openssl|openssh-server"
# ii  openssl  3.0.2-0ubuntu1.15  amd64  Secure Sockets Layer toolkit
sudo apt update && apt list --upgradable
sudo apt upgrade -y

rpm -q openssl
sudo dnf check-update
sudo dnf upgrade -y
```

What to say to the interviewer: *I check versions with* `dpkg -l` *or* `apt policy` *on Ubuntu and* `rpm -q` *on RHEL, list pending updates with* `apt list --upgradable` *or* `dnf check-update`*, and apply them with* `apt upgrade` *or* `dnf upgrade`*. I record the before-versions and patch history so I can roll back if needed.*

Q19. How do you find the OS version on Linux?

In simple words: Like checking the model and year written inside a car's door frame before ordering spare parts - you need to know the exact version before installing or patching anything.

Detailed answer

- `cat /etc/os-release` -> works on almost every modern distro (NAME, VERSION_ID, PRETTY_NAME). Best first choice, and easy to use in scripts (`. /etc/os-release; echo $ID $VERSION_ID`).
- `hostnamectl` -> OS, kernel and architecture in one view (systemd systems).
- `lsb_release -a` -> Debian/Ubuntu (may need the `lsb-release` package).
- `cat /etc/redhat-release` or `/etc/system-release` -> RHEL, CentOS, Rocky, Amazon Linux.
- `uname -r` -> **kernel** version only (not the distro); `uname -a` for everything including architecture.

Example:

```
cat /etc/os-release | grep -E "^(NAME|VERSION_ID)="
# NAME="Amazon Linux"
# VERSION_ID="2023"
hostnamectl | grep -E "Operating System|Kernel"
uname -r            # 6.1.100-1.amzn2023.x86_64
. /etc/os-release && [ "$ID" = "ubuntu" ] && sudo apt update || sudo dnf makecache
```

What to say to the interviewer: *I use* `cat /etc/os-release` *because it works on nearly every distro and is easy to parse in scripts, or* `hostnamectl` *for a quick summary.* `uname -r` *gives the kernel version, which is different from the distribution version.*

Q20. How do you search for a specific file inside subfolders?

In simple words: `find` walks through every drawer of the cabinet looking for the file right now; `locate` checks a pre-printed index, which is instant but may be a day old.

Detailed answer

- `find /path -type f -name "app.conf"` -> exact name, recursive through all subfolders.
- `-iname` -> ignore case; wildcards need quotes: `-name "*.log"`.
- `-type d` for directories; `-maxdepth 3` to limit how deep it goes.
- `2>/dev/null` hides "Permission denied" noise; `-xdev` stays on one filesystem.
- `locate app.conf` -> very fast, searches a database updated daily; run `sudo updatedb` to refresh it (package `mlocate`/`plocate`).
- `fd app.conf` -> a modern, faster find alternative, if installed.
- Act on results: `-exec ls -l {} +` or `-delete` (careful).

Example:

```
find /etc -type f -name "nginx.conf" 2>/dev/null
find / -xdev -iname "*docker-compose*.yml" 2>/dev/null
find /opt -maxdepth 3 -type d -name "releases"
sudo updatedb && locate -i jenkins.war
```

> ***What to say to the interviewer:*** *I use* `find /path -type f -name "file"`*, with* `-iname` *for case-insensitive search and* `2>/dev/null` *to hide permission errors. For a quick system-wide lookup I use* `locate`*, remembering its database has to be refreshed with* `updatedb`*.*

Q21. How do you check which ports are open or listening on a Linux server?

> **In simple words:** `ss` tells you which doors in the building are unlocked from the inside. `nmap` from another machine tells you which doors you can actually reach from the street, after the security guards (firewall, Security Group) have had their say.

Detailed answer

- `ss -tulnp` -> TCP/UDP, listening sockets, numeric ports, and the owning process (needs sudo for `-p`). This is the modern replacement for `netstat -tulnp`.
- `sudo lsof -i :8080` -> which process owns a specific port.
- `ss -tn state established '( dport = :5432 )'` -> active connections to a port.
- Bind address matters: `127.0.0.1:8080` is reachable only locally, `0.0.0.0:8080` or `[::]:8080` on all interfaces.
- **Listening is not the same as reachable:** test from outside with `nc -zv host 8080` or `nmap -p 8080 host`, then check firewalld/ufw/iptables and the cloud Security Group/NACL.

Example:

```
sudo ss -tulnp
# Netid State  Local Address:Port  Process
# tcp   LISTEN 0.0.0.0:22          users:(("sshd",pid=812,fd=3))
# tcp   LISTEN 127.0.0.1:5432      users:(("postgres",pid=990,fd=6))   <- local only
# tcp   LISTEN *:80                users:(("nginx",pid=1204,fd=6))

sudo lsof -i :80
nc -zv 10.0.2.15 80            # from another host: is it reachable?
sudo firewall-cmd --list-ports || sudo ufw status
```

> ***What to say to the interviewer:*** *I use* `ss -tulnp` *to see listening ports, the bind address and the owning process, and* `lsof -i :port` *for one specific port. I remember that listening on 127.0.0.1 means local only, and that a listening port may still be blocked, so I test from another host with* `nc -zv` *or nmap and then check the host firewall and the Security Group.*

Q22. What is the difference between curl and wget?

> **In simple words:** curl is a Swiss-army knife for talking to any web API and seeing exactly what comes back. wget is a download manager: point it at a file or a whole site and it fetches everything, even resuming if the line drops.

Detailed answer

Feature	curl	wget
Main purpose	Transfer data, test and call APIs	Download files and mirror sites
Default output	Prints to stdout	Saves to a file
HTTP methods and headers	Any method (`-X`), headers `-H`, data `-d`	Mainly GET/POST
Recursive download	No	Yes (`-r`, `--mirror`)
Resume download	`-C -`	`-c`
Protocols	Many (HTTP, FTP, SFTP, SMTP, LDAP ...)	HTTP, HTTPS, FTP
Library	libcurl, used by many apps	Standalone tool

- Use **curl** for health checks, REST APIs, debugging TLS or headers (`-v`, `-I`) and timing (`-w`).
- Use **wget** to download installers or artifacts, resume big downloads, or run background downloads (`-b`).

Example:

```
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://api.example.com/health
curl -X POST -H 'Content-Type: application/json' -d '{"name":"test"}' https://api.example.com/items
curl -vI https://example.com            # headers + TLS handshake details

wget -c https://releases.example.com/app-1.4.2.tar.gz     # resume if interrupted
wget -q -O - https://example.com/install.sh | head        # print to stdout like curl
```

> ***What to say to the interviewer:*** *curl is my tool for APIs and troubleshooting: any HTTP method, custom headers, verbose TLS output and timing, with output to stdout. wget is better for downloading files, resuming large downloads and mirroring sites recursively. In scripts I use curl with* `-fsS` *and a timeout for health checks.*

Advanced

Q23. Write a shell script to verify whether SSH connectivity to a server (or list of servers) is working.

> **In simple words:** Like a security guard walking down a corridor and trying each door with the key - noting which open, which are locked, and which are not answering at all.

Detailed answer

Good script qualities: non-interactive (`BatchMode=yes` so it never waits for a password), a short `ConnectTimeout`, a separate TCP port check to tell "network down" from "auth failed", clear exit codes, and a summary for cron/CI.

Example:

```
#!/usr/bin/env bash
# Usage: ./check_ssh.sh servers.txt [user]
set -uo pipefail
FILE="${1:?Usage: $0 <servers_file> [user]}"
USER_NAME="${2:-ubuntu}"
FAILED=0

while read -r HOST; do
  [ -z "$HOST" ] && continue
  if ! nc -z -w 3 "$HOST" 22 2>/dev/null; then
    echo "[DOWN]  $HOST - port 22 not reachable (network/SG/firewall)"
    FAILED=$((FAILED+1)); continue
  fi
  if ssh -n -o BatchMode=yes -o ConnectTimeout=5 \
         -o StrictHostKeyChecking=accept-new \
         "$USER_NAME@$HOST" 'hostname' >/dev/null 2>&1; then
    echo "[OK]    $HOST"
  else
    echo "[AUTH]  $HOST - port open but SSH login failed (key/user/permissions)"
    FAILED=$((FAILED+1))
  fi
done < "$FILE"

echo "Failed: $FAILED"
exit $(( FAILED > 0 ? 1 : 0 ))
```

Note `ssh -n` stops ssh from eating the rest of the input file inside the while loop - a classic bug.

> ***What to say to the interviewer:*** *My script loops over hosts, first checks port 22 with* `nc -z` *to separate network issues from authentication issues, then runs* `ssh -o BatchMode=yes -o ConnectTimeout=5 user@host hostname`*. It reports OK/DOWN/AUTH per host and exits non-zero if any fail, so it works in cron or a pipeline.*

Q24. How can you access a Windows C: drive from an Ubuntu system (dual boot or mounted disk)?

> **In simple words:** The Windows disk is like a filing cabinet in a different language. Ubuntu needs the translator (NTFS driver) and then you just open the drawer at a folder you choose.

Detailed answer

1. Find the partition: `lsblk -f` or `sudo fdisk -l` -> look for the `ntfs` partition (e.g. `/dev/sda3` or `/dev/nvme0n1p3`).
2. Create a mount point: `sudo mkdir -p /mnt/windows`.
3. Mount it: `sudo mount -t ntfs3 /dev/sda3 /mnt/windows` (kernel 5.15+ `ntfs3` driver) or `-t ntfs-3g` (install `ntfs-3g`). Use `-o ro` for read-only to be safe.
4. Make it permanent via `/etc/fstab` using the UUID from `blkid`.

Common gotcha: if Windows was shut down with **Fast Startup / hibernation**, the disk is marked "in use" and mounts read-only or fails. Fix by disabling Fast Startup in Windows or doing a full shutdown; `ntfsfix` can clear the dirty flag only as a last resort. BitLocker-encrypted drives need `dislocker` and the recovery key.

Example:

```
lsblk -f
# nvme0n1p3 ntfs  Windows  1A2B3C4D5E6F7788
sudo mkdir -p /mnt/windows
sudo mount -t ntfs3 -o ro /dev/nvme0n1p3 /mnt/windows
ls /mnt/windows/Users

# /etc/fstab (permanent)
UUID=1A2B3C4D5E6F7788  /mnt/windows  ntfs3  defaults,uid=1000,gid=1000,nofail  0  0
```

What to say to the interviewer: *I identify the NTFS partition with* `lsblk -f`*, create a mount point and mount it with the* `ntfs3` *or* `ntfs-3g` *driver, and add it to /etc/fstab by UUID for persistence. If it mounts read-only, it is usually Windows Fast Startup or hibernation, which must be disabled first.*

Q25. How do you troubleshoot network latency step by step?

In simple words: A slow pizza delivery could be slow ordering (DNS), traffic on the road (network), a slow door check (TLS), or a slow kitchen (application). You time each stage to find the real culprit.

Detailed answer

1. **Measure precisely where time goes** with `curl -w`: DNS lookup, TCP connect, TLS handshake, time to first byte (TTFB), total.
2. **DNS slow** -> resolver issue, low TTL, too many search domains (`ndots` in Kubernetes).
3. **Connect slow** -> network distance, packet loss, congested NAT/firewall; check with `mtr` for loss per hop.
4. **TLS slow** -> no session reuse, huge certificate chains, CPU-starved terminator.
5. **TTFB slow but connect fast** -> the server/app/database is slow, not the network.
6. Check the host: `ss -s` (connection counts, TIME_WAIT), retransmits via `netstat -s | grep -i retrans`, NIC errors with `ip -s link`.
7. Compare from different locations (same subnet vs internet) to isolate the segment.
8. Look at p95/p99 not just averages - tail latency hides behind a good average.

Example:

```
curl -o /dev/null -s -w \
 "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} \
ttfb:%{time_starttransfer} total:%{time_total}\n" https://api.example.com/health
# dns:0.004 connect:0.021 tls:0.058 ttfb:1.842 total:1.845   <- app is slow, not network

mtr -rwc 50 api.example.com
netstat -s | grep -i retrans
```

What to say to the interviewer: *I break the request into DNS, TCP connect, TLS and time-to-first-byte using* `curl -w`*, which immediately tells me whether it is a network or an application problem. Then I use* `mtr` *for packet loss, retransmit counters and connection stats on the host, and always look at p95/p99 rather than averages.*

Q26. How do you approach troubleshooting and debugging automation (shell) scripts?

In simple words: Debugging a script is like replaying a recipe step by step with a narrator saying out loud what is being done, so you catch the exact step where it went wrong.

Detailed answer

- Run with trace: `bash -x script.sh` or put `set -x` around a suspect block - prints every command with variables expanded.
- Use strict mode `set -euo pipefail` so errors are not silently ignored.
- Add a `trap` on ERR to print the failing line number.
- Lint with **shellcheck** - catches unquoted variables, wrong tests, etc. (add it to CI).
- Log with timestamps to a file; separate stdout and stderr.
- Reproduce in the same environment: cron and CI have a different `PATH`, user and working directory than your shell.
- Check exit codes (`echo $?`) of each external command.
- Make scripts idempotent (safe to re-run) and test with a `--dry-run` flag first.

Example:

```
#!/usr/bin/env bash
set -euo pipefail
trap 'echo "ERROR at line $LINENO: $BASH_COMMAND (exit $?)" >&2' ERR
log() { echo "$(date '+%F %T') $*" | tee -a /var/log/deploy.log; }

log "Starting backup"
set -x
tar -czf "/backup/app-$(date +%F).tar.gz" /opt/app
set +x
log "Done"
```

```
shellcheck deploy.sh
# SC2086: Double quote to prevent globbing and word splitting.
```

What to say to the interviewer: *I run the script with* `bash -x`*, use* `set -euo pipefail` *and an ERR trap to print the failing line, and lint it with shellcheck in CI. I also reproduce in the same context it runs in, like cron or the CI agent, because PATH and user differences cause most "works on my machine" script failures.*

Q27. Can you share an example of a complex automation script you have written?

In simple words: Interviewers want a short story: what boring or risky manual task existed, what your script did automatically, and what it saved.

Detailed answer

Use the structure **Problem -> What the script does -> Safety features -> Result**. A good Bash example: an automated disk cleanup and alert script for monitoring servers.

- Problem: small monitoring servers repeatedly hit 90%+ disk from journal logs and old exports.

- Script: checks usage with `df`, if above threshold vacuums journald, deletes rotated logs older than N days, removes old temp exports, re-checks usage and posts a message to a Teams/Slack webhook with before/after numbers.
- Safety: dry-run mode, only touches whitelisted paths, `flock` to prevent overlaps, logs every action.
- Result: no more manual 2 AM cleanups; alert only when cleanup is not enough.

Example:

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD=85; WEBHOOK="${WEBHOOK_URL:?}"; DRY="${DRY_RUN:-false}"
usage() { df --output=pcent / | tail -1 | tr -dc '0-9'; }
run() { [ "$DRY" = true ] && echo "DRY: $*" || eval "$@"; }

BEFORE=$(usage)
[ "$BEFORE" -lt "$THRESHOLD" ] && exit 0

run "journalctl --vacuum-size=300M"
run "find /var/log -name '*.gz' -mtime +7 -delete"
run "find /opt/exports -type f -mtime +14 -delete"
AFTER=$(usage)

MSG="$(hostname): disk ${BEFORE}% -> ${AFTER}%"
[ "$AFTER" -ge "$THRESHOLD" ] && MSG="$MSG - STILL HIGH, manual action needed"
curl -s -H 'Content-Type: application/json' -d "{\"text\":\"$MSG\"}" "$WEBHOOK"
```

> ***What to say to the interviewer:*** *One example is a disk-cleanup automation for our monitoring servers: it checks disk usage, vacuums journald and deletes old rotated logs and exports from whitelisted paths, re-measures, and posts before/after numbers to a Teams webhook. It has a dry-run mode and flock protection, and it removed a recurring manual task while only alerting us when cleanup was not enough.*

Q28. Write a shell script to find prime numbers (e.g. all primes up to N).

> **In simple words:** A prime number can only be divided evenly by 1 and itself. The script is like testing each number with a set of keys (divisors) - if any key fits, it is not prime. You only need to try keys up to the square root, because any bigger key would have a smaller partner key you already tried.

Detailed answer

Why interviewers ask it: it is not really about maths. It checks whether you can write working Bash with **argument validation**, **loops** (`for`/`while`), **conditionals** (`if`, `[[ ]]`), **arithmetic** (`$(( ))`, `(( ))`, modulo `%`), functions and return codes - the same building blocks used in real automation scripts.

Key points to say while writing it:

- Validate input: exactly one argument, and it must be a positive integer (regex `^[0-9]+$`).
- Numbers below 2 are not prime; 2 is the only even prime.
- Only test odd divisors from 3 up to sqrt(n) -> check `i * i <= n` instead of calling a sqrt tool (Bash has integer maths only).
- Use a function that returns 0 (prime) or 1 (not prime) so it reads cleanly in an `if`.

- Bash is fine for small N; for very large ranges a Sieve of Eratosthenes or another language is faster - mentioning this shows judgement.

Example:

```
#!/usr/bin/env bash
# Usage: ./primes.sh <N>   -> prints all primes from 2 to N
set -euo pipefail

usage() { echo "Usage: $0 <positive integer N>" >&2; exit 1; }

[ $# -eq 1 ] || usage
N="$1"
[[ "$N" =~ ^[0-9]+$ ]] || { echo "Error: '$N' is not a positive integer" >&2; exit 2; }

is_prime() {
  local n=$1 i
  (( n < 2 )) && return 1
  (( n == 2 )) && return 0
  (( n % 2 == 0 )) && return 1
  for (( i = 3; i * i <= n; i += 2 )); do
    (( n % i == 0 )) && return 1
  done
  return 0
}

primes=()
for (( num = 2; num <= N; num++ )); do
  if is_prime "$num"; then primes+=("$num"); fi
done

echo "Primes up to $N: ${primes[*]:-none}"
echo "Count: ${#primes[@]}"
```

```
$ ./primes.sh 30
Primes up to 30: 2 3 5 7 11 13 17 19 23 29
Count: 10

$ ./primes.sh abc
Error: 'abc' is not a positive integer

$ ./primes.sh
Usage: ./primes.sh <positive integer N>
```

> ***What to say to the interviewer:*** *I validate that exactly one positive integer is passed, then loop from 2 to N and test each number with an is_prime function that rejects numbers below 2 and even numbers, and checks odd divisors only while i times i is less than or equal to n. It demonstrates Bash loops, conditionals, arithmetic with double parentheses and return codes, and for very large N I would mention a sieve or a faster language.*

Q29. How do you extend a partition or filesystem without unmounting it?

> **In simple words:** Like adding extra rooms to a house while people are still living in it - first make the plot bigger (disk/volume), then move the wall (partition), then tell the house it now has more floor space (filesystem).

Detailed answer

Both ext4 and XFS can be **grown online** (while mounted). Shrinking XFS is impossible, and shrinking ext4 needs an unmount.

Plain partition (common on EC2):

1. Grow the disk: modify the EBS volume size in AWS (no downtime).
2. `lsblk` -> confirm the disk is bigger than the partition.
3. `sudo growpart /dev/nvme0n1 1` -> grow partition 1.
4. Grow the filesystem: `resize2fs` for ext4, `xfs_growfs /mountpoint` for XFS (check with `df -Th`).

LVM:

1. Add space: grow the disk + `pvresize`, or add a new disk with `pvcreate` + `vgextend`.
2. `lvextend -r -L +20G /dev/vg0/data` -> `-r` also resizes the filesystem in the same step.

Take a snapshot first - a resize is safe but not undoable.

Example:

```
lsblk; df -Th /
sudo growpart /dev/nvme0n1 1
sudo resize2fs /dev/nvme0n1p1        # ext4
sudo xfs_growfs /                    # XFS (Amazon Linux default)

# LVM
sudo pvresize /dev/nvme1n1
sudo lvextend -r -L +20G /dev/vg_data/lv_data
```

Also asked as: Add 50GB to /opt using LVM without any downtime.

Full chain for "add 50GB to /opt with LVM, no downtime":

1. `df -hT /opt` -> find the LV (e.g. `/dev/mapper/vg_app-opt`) and the filesystem type.
2. `vgs` -> is there at least 50G in the `VFree` column of the volume group?
3. If not: attach a new disk, then `pvcreate /dev/nvme2n1` and `vgextend vg_app /dev/nvme2n1` (or grow the existing disk and run `pvresize`).
4. `lvextend -r -L +50G /dev/vg_app/opt` -> `-r` grows the filesystem too. Or do it in two steps: `lvextend -L +50G ...`, then `xfs_growfs /opt` (XFS, uses the mount point) or `resize2fs /dev/vg_app/opt` (ext4).
5. `df -h /opt` to verify. Everything stays mounted and no application restart is needed.

> ***What to say to the interviewer:*** *For a normal partition I grow the EBS volume, run* `growpart`*, then* `resize2fs` *for ext4 or* `xfs_growfs` *for XFS, all while mounted. With LVM I extend the physical volume or volume group and run* `lvextend -r`*, which grows the logical volume and filesystem together; I always snapshot first.*

Q30. What are the steps to add a new disk to a Linux server?

> **In simple words:** A new hard disk is like an empty new cupboard. You divide it into shelves (partition), fit the shelving system (filesystem), place it in a room people can reach (mount point), and write it into the house plan so it is still there after a restart (fstab).

Detailed answer

1. Attach the disk (EBS attach, or physically) and find it: `lsblk` (new device e.g. `/dev/nvme1n1` with no mount point).
2. Optional partition: `parted /dev/nvme1n1 mklabel gpt mkpart primary 0% 100%` (or use the whole disk / LVM).
3. Create a filesystem: `mkfs.xfs` or `mkfs.ext4`.
4. Create a mount point and mount: `mkdir /data && mount /dev/nvme1n1p1 /data`.
5. Make it permanent in `/etc/fstab` using the **UUID** (device names like nvme1n1 can change after reboot), with `nofail` so a missing disk does not stop boot.
6. Test the fstab entry with `mount -a` BEFORE rebooting - a bad fstab line can leave the server unbootable.
7. Set ownership/permissions for the app user.

Example:

```
lsblk
sudo parted -s /dev/nvme1n1 mklabel gpt mkpart primary xfs 0% 100%
sudo mkfs.xfs /dev/nvme1n1p1
sudo mkdir -p /data
sudo blkid /dev/nvme1n1p1        # UUID="3f2c..."
echo 'UUID=3f2c-...  /data  xfs  defaults,nofail  0 2' | sudo tee -a /etc/fstab
sudo mount -a && df -h /data
```

What to say to the interviewer: *I identify the new disk with* `lsblk`*, partition it with parted, create an XFS or ext4 filesystem, mount it on a directory, and add it to /etc/fstab by UUID with* `nofail`*. I always validate with* `mount -a` *before any reboot, because a broken fstab entry can stop the server booting.*

Q31. How do you create and mount a swap file?

In simple words: Swap is an overflow parking lot on disk. When RAM (the main car park) is full, less-used data is parked there - slower to fetch, but better than the OOM killer throwing things out.

Detailed answer

1. Create the file: `fallocate -l 2G /swapfile` (or `dd` if fallocate is not supported, e.g. on some XFS setups).
2. Lock it down: `chmod 600 /swapfile` (swap can contain secrets from memory).
3. Format as swap: `mkswap /swapfile`.
4. Enable: `swapon /swapfile`; verify with `swapon --show` / `free -h`.
5. Persist: add `/swapfile none swap sw 0 0` to `/etc/fstab`.
6. Tune `vm.swappiness` (e.g. 10) so the kernel prefers RAM and only swaps under pressure.

Swap is a safety buffer, not a fix for too little RAM - heavy swapping makes everything slow. Kubernetes nodes traditionally run with swap off (kubelet requires it unless swap support is explicitly configured).

Example:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf && sudo sysctl --system
free -h
```

What to say to the interviewer: *I create a file with fallocate, set it to 600, run mkswap and swapon, add it to /etc/fstab, and lower swappiness so it is only used under pressure. It is a buffer against OOM kills on small servers, not a replacement for right-sizing, and I keep swap off on Kubernetes nodes.*

Q32. How do you find which processes are writing to a file in real time?

In simple words: A shared notebook keeps filling up and you want to know who is writing in it. You can look at who is holding it right now (lsof/fuser), watch it live (inotifywait), or install a CCTV camera that records every writer (auditd).

Detailed answer

- `lsof /var/log/app.log` -> processes that currently have the file open (the `FD` column ends in `w` or `u` for write).
- `fuser -v /var/log/app.log` -> PIDs using the file (also works on mount points: `fuser -vm /data`).
- `inotifywait -m /var/log/app.log` (package `inotify-tools`) -> live stream of modify/open/close events. It shows WHAT happens, not WHO.
- `auditctl -w /etc/nginx/nginx.conf -p wa -k nginx_conf` -> audit rule recording writes and attribute changes WITH the PID, user and command; read with `ausearch -k nginx_conf`. Best for "who keeps changing this config?"
- For a known process, `strace -f -e trace=write -p <PID>` shows its writes live.

Example:

```
sudo lsof /var/log/app.log
# COMMAND  PID  USER  FD  TYPE ... NAME
# java    2143  app   12w  REG ... /var/log/app.log
sudo fuser -v /var/log/app.log
inotifywait -m -e modify,close_write /etc/nginx/nginx.conf
sudo auditctl -w /etc/nginx/nginx.conf -p wa -k nginx_conf
sudo ausearch -k nginx_conf -i | tail
```

What to say to the interviewer: *For who has it open right now I use* `lsof` *or* `fuser`*; to watch changes live I use* `inotifywait`*; and to record exactly which user and process modify it over time I add an auditd watch rule and query it with* `ausearch`*. For one known process,* `strace -e trace=write` *shows its writes in real time.*

Q33. What is the fastest way to copy a huge file (or many files) across servers?

(also asked as: How do you transfer a large file securely between servers?)

In simple words: Moving house - you can carry boxes one by one (scp), use a van that only moves boxes that changed and resumes if it stops (rsync), pack everything tightly first (compression), or hire several vans at once (parallel).

Detailed answer

Method	When to use
`rsync -avP`	default choice: resumable (`--partial`), progress, only sends changes
`scp` / `sftp`	simple one-off copy, not resumable
tar piped over ssh	millions of small files - one stream instead of per-file overhead
compression (`rsync -z`, `zstd`)	slow network + compressible data; skip it for already-compressed files or fast LANs (CPU becomes the bottleneck)
parallel streams (`parallel` + rsync, `bbcp`)	high-latency, high-bandwidth links where one TCP stream cannot fill the pipe
On AWS: via **S3**	`aws s3 cp` uses multipart parallel uploads, then download on the other side; also works across regions/accounts

Tips: use a light SSH cipher only if allowed, run inside `tmux`, verify with `sha256sum` on both sides.

Making it secure: always go over SSH (`scp`, `rsync -e ssh`, `sftp`) with key-based auth - never plain FTP/rsync daemon on untrusted networks. Use a dedicated low-privilege transfer user, and a jump host (`-J bastion`) instead of opening port 22 widely. On S3, use a private bucket with SSE-KMS encryption, a VPC endpoint and short-lived pre-signed URLs or IAM roles. Sensitive files can be encrypted before sending (`gpg -c` or `age`). Finally, prove integrity: compare `sha256sum` on both ends.

Example:

```
rsync -avP --partial /data/dump.sql.gz user@10.0.2.20:/backup/
tar -cf - /data/images | zstd -T0 | ssh user@10.0.2.20 "zstd -d | tar -xf - -C /restore"
aws s3 cp /data/dump.sql.gz s3://my-transfer-bucket/     # multipart, parallel
sha256sum /data/dump.sql.gz      # compare with the destination
```

What to say to the interviewer: *My default is* `rsync -avP` *because it is resumable and only sends differences; for millions of small files I stream* `tar` *over ssh, and I only compress when the network is slower than the CPU. On AWS I often copy through S3 because multipart uploads are parallel, and I always verify with checksums.*

Q34. How do you analyse disk I/O performance on Linux?

In simple words: The disk is a single checkout counter. You check how busy the cashier is (%util), how long each customer waits (await), how long the queue is, and who is buying the most (iotop).

Detailed answer

- `iostat -xz 1` -> per device: `r/s`, `w/s`, `rkB/s`, `wkB/s`, `await` (ms per request, including queue), `aqu-sz` (queue length), `%util`. High await + high queue = the disk is the bottleneck. (%util near 100% is less meaningful on SSD/NVMe, which serve requests in parallel.)

- `vmstat 1` -> wa column (CPU waiting on IO) and b (processes blocked on IO).
- `iotop -o` / `pidstat -d 1` -> which process is doing the IO.
- `top` -> processes in `D` state are stuck waiting on IO.
- Benchmark capability with `fio` (on a test file, never over live data).
- On AWS also check EBS limits: volume IOPS/throughput (gp3 baseline 3000 IOPS / 125 MB/s), burst balance on gp2, and the instance's EBS bandwidth limit in CloudWatch.

Example:

```
iostat -xz 1 3
# Device  r/s   w/s   wkB/s   await  aqu-sz  %util
# nvme1n1 12.0  950.0 60800.0  38.2   36.1   99.8   <- saturated
sudo iotop -o
pidstat -d 1 5
fio --name=randwrite --filename=/data/fio.test --rw=randwrite --bs=4k --size=1G \
    --iodepth=32 --ioengine=libaio --direct=1 --runtime=30 --time_based
```

> ***What to say to the interviewer:*** *I use* `iostat -x` *to read await, queue size and utilisation per device,* `vmstat` *for IO wait and blocked processes, and* `iotop` *or* `pidstat -d` *to find the process causing it. On AWS I compare against the EBS volume's IOPS and throughput limits in CloudWatch and use* `fio` *on a test file to benchmark.*

Q35. How do you roll back a bad patch or package update?

> **In simple words:** Before repainting a wall you take a photo and keep the old paint tin. If the new colour is wrong you repaint with the old tin (downgrade) or, if it is a disaster, restore the whole room from the photo (snapshot).

Detailed answer

Options, from lightest to strongest:

1. **RHEL/Amazon Linux:** `dnf history` (or `yum history`) lists every transaction -> `dnf history undo <ID>` reverts that transaction, or `dnf downgrade pkg`.
2. **Ubuntu/Debian:** no built-in undo. Find the old version in `/var/log/apt/history.log`, then `apt install pkg=<old-version>` (if still in the repo or apt cache), and `apt-mark hold pkg` so it is not re-upgraded.
3. **Kernel:** boot the previous kernel from GRUB (kept installed by default) and set it as default.
4. **Whole server:** restore the **EBS snapshot / AMI** taken before patching - the most reliable rollback.
5. **Immutable infra:** redeploy the previous AMI/image version.

That is why the rule is: snapshot or AMI before patching production, and patch staging first.

Example:

```
sudo dnf history
# ID | Command line  | Date and time    | Action(s) | Altered
#  42 | upgrade -y    | 2026-09-20 02:00 | Upgrade   |   37
sudo dnf history undo 42

grep -A3 "Upgrade:" /var/log/apt/history.log | grep nginx
sudo apt install nginx=1.18.0-6ubuntu14.4
sudo apt-mark hold nginx
```

> ***What to say to the interviewer:*** *On RHEL-based systems I use* `dnf history undo` *for the transaction, on Ubuntu I reinstall the exact previous version with* `apt install pkg=version` *and hold it, and for kernels I boot the older kernel from GRUB. The safest rollback is the EBS snapshot or AMI I take before every production patch.*

Q36. How do you apply a kernel patch, reboot safely, and keep the old kernel available?

> **In simple words:** Changing the engine of a car. You keep the old engine in the garage, so if the new one will not start, you drop the old one back in and drive away.

Detailed answer

1. Snapshot/AMI first; drain the node or take it out of the load balancer.
2. Install the new kernel: `apt install linux-image-<ver>` / `apt full-upgrade`, or `dnf update kernel`. Package managers install kernels **side by side** - the old one stays.
3. Check the new kernel is registered in GRUB: `grubby --default-kernel` (RHEL) or `/boot/grub/grub.cfg` (Ubuntu); keep at least 2 kernels (`installonly_limit=3` in dnf.conf; do not `autoremove` the previous one right away).
4. Reboot in the maintenance window; verify with `uname -r` and service health checks.
5. If it fails: pick the old kernel from the GRUB menu (EC2 serial console helps here), then set it as the default (`grubby --set-default` or `GRUB_DEFAULT` + `update-grub`).
6. Check whether a reboot is needed at all: `/var/run/reboot-required` (Ubuntu), `needs-restarting -r` (RHEL).

Example:

```
uname -r
sudo dnf update kernel -y
sudo grubby --info=ALL | grep -E "^kernel"
sudo needs-restarting -r
sudo reboot
uname -r                                        # verify new kernel
sudo grubby --set-default /boot/vmlinuz-6.1.100-1.amzn2023.x86_64   # roll back if needed
```

> ***What to say to the interviewer:*** *I snapshot and drain the node, install the new kernel, which sits alongside the old one, confirm GRUB has both, reboot in a maintenance window and verify with* `uname -r` *and health checks. If it misbehaves I boot the previous kernel from GRUB and set it as default, and I keep at least two kernels installed.*

Q37. What is live kernel patching?

In simple words: Fixing a tyre while the car is still driving. You fix the dangerous hole now without stopping, and change the whole tyre at the next pit stop.

Detailed answer

Live patching applies **security fixes to the running kernel in memory** without a reboot, by redirecting vulnerable functions to patched versions.

- Tools: **kpatch** (Red Hat), **kGraft** (SUSE, merged into upstream livepatch), **Canonical Livepatch** (Ubuntu Pro), **Ksplice** (Oracle).
- Good for: critical CVEs on servers where reboots are expensive (databases, stateful nodes), buying time until the next maintenance window.
- Limits: only for selected security fixes, not full kernel upgrades; the fix is lost on reboot unless the new kernel package is also installed; you still need a real reboot eventually.
- In cloud/Kubernetes environments, rolling node replacement (new AMI) often makes live patching unnecessary.

Example:

```
# Ubuntu Pro
sudo pro enable livepatch
canonical-livepatch status --verbose
# RHEL
sudo dnf install kpatch-patch-$(uname -r | sed 's/\.x86_64//;s/\./_/g')
kpatch list
```

What to say to the interviewer: *Live patching - with kpatch, Canonical Livepatch, kGraft or Ksplice - patches critical kernel vulnerabilities in memory without a reboot. I use it to protect hard-to-reboot servers until a planned maintenance window, but it does not replace installing the new kernel and rebooting eventually.*

Q38. How do you patch a fleet of servers in production?

In simple words: Vaccinating a whole city: you test the vaccine on a small group first, do it neighbourhood by neighbourhood at quiet times, keep a record of who got it, and stop immediately if the first group has problems.

Detailed answer

1. **Inventory and compliance**: know which servers need which patches (SSM Patch Manager compliance, Inspector findings, CVE feeds).
2. **Tooling**: **AWS SSM Patch Manager** (patch baselines, patch groups by tag, maintenance windows, compliance reports) or **Ansible** playbooks (`serial:` batches) for non-AWS/hybrid.
3. **Order**: dev -> staging -> a **canary batch** in prod (e.g. 10%) -> the rest in batches, never all at once or both nodes of an HA pair together.
4. **Safety**: pre-patch snapshots/AMIs, drain from load balancer first, health checks after each batch, automatic stop if failures exceed a threshold (`max_fail_percentage` in Ansible, error threshold in SSM).
5. **Maintenance windows** and change approval; communicate with teams.

6. **Immutable alternative**: rebake a patched AMI (Packer / EC2 Image Builder), roll it out through the Auto Scaling Group instance refresh, or replace EKS node groups - no in-place patching at all.
7. Report compliance afterwards.

Example:

```
# Ansible: rolling patch, 20% at a time, stop if >10% fail
- hosts: web
  serial: "20%"
  max_fail_percentage: 10
  become: true
  tasks:
    - name: Upgrade all packages
      ansible.builtin.dnf: { name: "*", state: latest }
    - name: Reboot if needed
      ansible.builtin.reboot: { reboot_timeout: 600 }
    - name: Health check
      ansible.builtin.uri: { url: "http://localhost/health", status_code: 200 }
```

> ***What to say to the interviewer:*** *I patch in stages - dev, staging, then a prod canary batch and the rest in rolling batches during maintenance windows - using SSM Patch Manager or Ansible with serial batches, pre-patch snapshots, load balancer draining and a failure threshold that stops the rollout. For immutable workloads I instead rebake a patched AMI and roll it out with an ASG instance refresh or node group replacement.*

Q39. In Nginx, how do you make a client stick to a specific server (sticky sessions)?

> **In simple words:** A regular customer always goes to the same bank teller because that teller has their file on the desk. Stickiness keeps sending them to that teller - but it is better to keep the file in a shared cabinet (Redis) so any teller can help.

Detailed answer

Options in open-source nginx:

- `ip_hash`: hashes the client IP (first 3 octets for IPv4) -> same server. Simple, but many users behind one corporate NAT or mobile carrier all land on one server, and it breaks when client IPs change.
- `hash $cookie_sessionid consistent;` (or any key like `$request_uri`): hash a session cookie or header. `consistent` uses ketama consistent hashing, so adding or removing a server only moves about 1/N of the clients.
- `sticky cookie` directive: nginx itself inserts a cookie naming the server - available only in **NGINX Plus** (commercial). HAProxy (`cookie SERVERID insert`) and AWS ALB (target group stickiness) provide it for free.

Why stickiness is a last resort: load becomes uneven, scaling in/out and deployments break sessions, and a failed server logs its users out. The better design is **stateless app servers with the session externalised** in Redis/Memcached/a DB or a signed token (JWT), so any server can serve any request. Use stickiness only for legacy apps or cache locality.

Example:

```
upstream app_sticky {
    hash $cookie_JSESSIONID consistent;   # or: ip_hash;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.13:8080 max_fails=3 fail_timeout=30s;
}
# NGINX Plus only:
# upstream app { server 10.0.1.11:8080; server 10.0.1.12:8080;
#                sticky cookie srv_id expires=1h path=/; }
```

What to say to the interviewer: *In open-source nginx I use* `ip_hash` *or, better,* `hash $cookie_<session> consistent` *so the same session always maps to the same server; the* `sticky cookie` *directive exists only in NGINX Plus. I prefer to avoid stickiness altogether by storing sessions in Redis or using tokens, because sticky sessions cause uneven load and lost sessions when a server fails.*

Q40. In Nginx, with sticky sessions or ip_hash, what happens if a server goes down?

In simple words: If your usual bank teller goes home sick, the manager sends you to another teller - but that teller does not have your file, so you must start again, unless the files are kept in a shared cabinet.

Detailed answer

- Open-source nginx uses **passive health checks**: when requests to a server fail (connection errors, timeouts, and optionally 5xx via `proxy_next_upstream`) `max_fails` times within `fail_timeout`, nginx marks it **unavailable for** `fail_timeout` seconds, then tries it again.
- With `ip_hash`/`hash`, clients mapped to the dead server are **remapped to other servers**. Their in-memory session is on the dead server, so they **lose their session** (logged out, cart empty) - unless sessions are shared (Redis) or replicated.
- The first request that hits the failure can still error or be slow (it has to time out), then `proxy_next_upstream` retries on another server for idempotent requests.
- `consistent` hashing limits the damage: only the failed server's clients move; with plain modulo hashing, removing a server reshuffles almost everyone.
- `down` is a manual flag for planned maintenance. With `ip_hash` it keeps the hash mapping for the other servers (clients of the down server are spread elsewhere), so use `down` instead of deleting the line.
- **Active health checks** (`health_check` in NGINX Plus, HAProxy, ALB) probe a URL in the background and remove a server BEFORE user requests fail, and put it back when it recovers.

Example:

```
upstream app {
    ip_hash;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.13:8080 down;          # planned maintenance
}
location / {
    proxy_pass http://app;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_next_upstream_tries 2;
    proxy_connect_timeout 2s;
}
```

What to say to the interviewer: *After* `max_fails` *failures within* `fail_timeout`*, nginx marks the server unavailable and remaps its clients to the remaining servers, so those users lose their session unless it is stored centrally in something like Redis. Consistent hashing means only that server's clients move,* down *is for planned maintenance, and active health checks in NGINX Plus, HAProxy or an ALB remove the server before users see errors.*

Q41. How does header management work in Nginx?

In simple words: Headers are the notes written on the outside of an envelope. As a reverse proxy, nginx rewrites the notes going to the backend (who really sent it, which address it was for) and adds security stickers on the replies going back to the browser.

Detailed answer

Request headers to the backend (`proxy_set_header`):

- `Host $host` -> otherwise nginx sends the upstream name, which breaks virtual hosts.
- `X-Real-IP $remote_addr` and `X-Forwarded-For $proxy_add_x_forwarded_for` -> the real client IP; without them the app only sees nginx's IP.
- `X-Forwarded-Proto $scheme` -> tells the app the original request was HTTPS (for correct redirects and secure cookies).

Response headers to the client (`add_header`):

- Security headers: `Strict-Transport-Security` (HSTS), `X-Content-Type-Options nosniff`, `X-Frame-Options`, `Content-Security-Policy`.
- Add `always`, otherwise the header is only sent on 2xx/3xx responses, not on errors.
- **Inheritance gotcha**: `add_header` (and `proxy_set_header`) directives are inherited from the parent block ONLY if the child block defines none of its own. One `add_header` inside a `location` silently drops all the server-level ones -> repeat them or use an `include` snippet.
- `proxy_hide_header X-Powered-By;` -> remove backend headers that leak versions; `server_tokens off;` hides the nginx version.

Other gotchas:

- `underscores_in_headers off` is the default, so headers like `api_key` are silently DROPPED. Use dashes or turn it on.
- Large cookies or JWTs -> `400 Request Header Or Cookie Too Large` -> raise `large_client_header_buffers 4 16k;`. On the response side, big upstream headers need `proxy_buffer_size`.

Example:

```
server {
    listen 443 ssl;
    server_name app.example.com;
    server_tokens off;
    large_client_header_buffers 4 16k;
    include /etc/nginx/snippets/security-headers.conf;   # reusable add_header block

    location /api/ {
        proxy_pass http://app_backend;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_hide_header X-Powered-By;
        include /etc/nginx/snippets/security-headers.conf;  # re-include: add_header here
        add_header Cache-Control "no-store" always;         # would drop parent headers
    }
}
# /etc/nginx/snippets/security-headers.conf
# add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
# add_header X-Content-Type-Options "nosniff" always;
# add_header X-Frame-Options "DENY" always;
```

What to say to the interviewer: *I use* `proxy_set_header` *to pass Host, X-Real-IP, X-Forwarded-For and X-Forwarded-Proto so the backend knows the real client and scheme, and* `add_header ... always` *for security headers like HSTS and nosniff. The classic gotchas are that a child block with its own add_header drops all inherited ones, that headers containing underscores are dropped by default, and that large cookies need* `large_client_header_buffers`*.*

Q42. How do you identify and manage parent-child process relationships in Linux?

In simple words: Processes form a family tree. Every process has a parent that created it. When a child finishes, the parent must sign off the paperwork (wait), otherwise the child becomes a zombie; if the parent dies first, the child is adopted by the grandparent of all processes (init/systemd).

Detailed answer

- Every process has a **PID** and a **PPID** (parent's PID). New processes are made with **fork()** (copy of the parent) followed by **exec()** (replace with a new program) - that is how a shell runs every command.
- See the tree: `ps -ef --forest`, `ps -o pid,ppid,pgid,sid,stat,cmd`, `pstree -p`, or `grep PPid /proc/<PID>/status`.
- **Orphans**: if a parent dies first, the child is re-parented to PID 1 (systemd/init) or to a **subreaper** (for example a systemd user manager or a container init like tini), which later reaps it.
- **Zombies**: a child that exited but whose parent has not called `wait()`; the parent receives `SIGCHLD` and should reap it (see Q58).
- **Process groups and sessions**: a pipeline shares a process group (PGID). `kill --<PGID>` signals the whole group - useful to stop a script AND all its children. `pkill -P <PPID>` kills the children of a parent.

- Detaching: `nohup cmd &` ignores hangup when you log out; `setsid cmd` starts a new session with no controlling terminal; for real services use systemd, which tracks all children in a cgroup (`systemctl status` shows the tree, `systemctl stop` kills them all).

Example:

```
pstree -p $(pgrep -o nginx)
# nginx(1201)-+-nginx(1202)
#             `-nginx(1203)
ps -o pid,ppid,pgid,stat,cmd --forest -g $(ps -o sid= -p $$)
grep -E "PPid|State" /proc/1202/status
kill -TERM -- -$(ps -o pgid= -p 4400 | tr -d ' ')   # stop the whole process group
pkill -TERM -P 4400                                  # stop only the children of 4400
setsid ./long_job.sh > job.log 2>&1 < /dev/null &
```

> ***What to say to the interviewer:*** *I look at PID and PPID with* `ps -ef --forest`*,* `ps -o pid,ppid,pgid,stat` *or* `pstree -p`*, knowing that processes are created by fork and exec, orphans are adopted by PID 1 or a subreaper, and zombies exist until the parent reaps them after SIGCHLD. To manage them I signal a whole process group with* `kill -- -PGID`*, use* `pkill -P` *for the children, and prefer systemd, which tracks every child in a cgroup.*

Q43. Write a script to convert human-readable file sizes (e.g. 1.5K, 20M, 3G) to bytes.

> **In simple words:** Like converting "1.5 kg" and "300 g" into grams so you can compare and add them. The script reads the unit letter, multiplies by the right number, and rejects nonsense like "5X".

Detailed answer

Know the two standards: **IEC** (binary) K = 1024, M = 1024^2, G = 1024^3 (what `ls -h`, `du -h` and `df -h` use), and **SI** (decimal) k = 1000, M = 1000^2 (what disk vendors and `df -H` use). Say which one you assume.

- Easiest: GNU `numfmt --from=iec 1.5K` -> 1536 (use `--from=si` for 1000-based).
- Pure Bash cannot do decimals (integer maths only), so use `awk` for the arithmetic.
- Handle: optional decimals, upper/lowercase units, optional `B`/`iB` suffix, no unit = bytes, invalid input -> error and non-zero exit.

Example:

```
#!/usr/bin/env bash
# Usage: ./to_bytes.sh 1.5K 20M 3G 512
set -uo pipefail

to_bytes() {
  local input="${1^^}"                       # uppercase: 1.5k -> 1.5K
  input="${input%IB}"; input="${input%B}"    # accept 1.5KiB, 20MB, 512B
  if [[ ! "$input" =~ ^([0-9]+(\.[0-9]+)?)([KMGT]?)$ ]]; then
    echo "invalid size: $1" >&2; return 1
  fi
  local num="${BASH_REMATCH[1]}" unit="${BASH_REMATCH[3]}" power=0
  case "$unit" in K) power=1 ;; M) power=2 ;; G) power=3 ;; T) power=4 ;; esac
  awk -v n="$num" -v p="$power" 'BEGIN { printf "%.0f\n", n * (1024 ^ p) }'
}

[ $# -ge 1 ] || { echo "Usage: $0 <size>..." >&2; exit 1; }
rc=0
for s in "$@"; do
  if b=$(to_bytes "$s"); then printf "%-8s = %s bytes\n" "$s" "$b"; else rc=2; fi
done
exit $rc
```

```
$ ./to_bytes.sh 1.5K 20M 3g 512 2T 5X
1.5K     = 1536 bytes
20M      = 20971520 bytes
3g       = 3221225472 bytes
512      = 512 bytes
2T       = 2199023255552 bytes
invalid size: 5X

$ numfmt --from=iec 1.5K 20M 3G       # one-liner alternative
1536
20971520
3221225472
$ numfmt --from=si 20M                # decimal units
20000000
```

> ***What to say to the interviewer:*** *The quick answer is* `numfmt --from=iec`*, or* `--from=si` *for 1000-based units. In a script I uppercase the input, strip an optional B or iB, validate it with a regex capturing the number and a K, M, G or T unit, and do the multiplication in awk because Bash has no floating point, returning an error for invalid input. I also state whether I assume 1024-based IEC or 1000-based SI units.*

Scenario-Based

Q44. An Ubuntu OS upgrade failed midway. How do you troubleshoot and recover?

> **In simple words:** It is like a house renovation stopped halfway - first make sure the house still stands, then finish or undo the half-done work carefully, and next time take photos (backups) before starting.

Detailed answer

1. **Before anything, snapshot** (EBS/VM snapshot) if the machine is still up - so recovery attempts are reversible.

2. If it still boots: finish the interrupted package work:

- `sudo dpkg --configure -a` -> configure half-installed packages.
- `sudo apt --fix-broken install` (`apt -f install`).
- `sudo apt update && sudo apt full-upgrade`.
- Check `/var/log/dist-upgrade/` and `/var/log/apt/term.log` for the actual error.

3. Common causes: disk full on `/` or `/boot` (old kernels), held packages, third-party PPAs, network interruption, dpkg lock left behind.

4. If it does **not** boot: at GRUB choose **Advanced options** -> an older kernel or recovery mode -> root shell -> remount rw (`mount -o remount,rw /`) and run the dpkg/apt fixes.

5. If GRUB itself is broken: boot a live USB (or on AWS, detach the root volume and attach it to a rescue instance), `chroot` into the disk, fix packages, `update-grub`/`grub-install`.

6. Last resort: restore from snapshot/backup, or rebuild (immutable infra makes this easy).

7. Prevention: test the upgrade on staging first, take a snapshot, run in `tmux`/`screen` so an SSH drop does not kill it, ensure free space.

Example:

```
sudo dpkg --configure -a
sudo apt --fix-broken install
sudo apt update && sudo apt full-upgrade
df -h / /boot
tail -50 /var/log/dist-upgrade/main.log

# Rescue via chroot (live USB or rescue EC2 with the volume attached)
sudo mount /dev/xvdf1 /mnt
for d in dev proc sys run; do sudo mount --bind /$d /mnt/$d; done
sudo chroot /mnt
dpkg --configure -a && apt -f install && update-grub
```

> ***What to say to the interviewer:*** *If the system still boots I run* `dpkg --configure -a` *and* `apt --fix-broken install`*, then read the dist-upgrade logs for the real cause, often a full disk or a third-party repo. If it will not boot I use an older kernel or recovery mode, or a live/rescue instance with chroot to repair, and restore from snapshot as a last resort; going forward I always snapshot and test upgrades on staging, running them in tmux.*

Q45. A Linux node shows high CPU usage even though all the pods on it appear healthy. What could be the reasons?

(also asked as: A service is consuming 100% CPU - how do you troubleshoot it? / High CPU usage is degrading performance - what do you do?)

> **In simple words:** The shop assistants (pods) look fine, but the building is still overheating. Something else is using the power - the building's own machinery, a hidden process, or even the neighbour sharing your electricity line (noisy neighbour on a VM).

Detailed answer

Step-by-step:

1. `top`/`htop` on the node -> sort by CPU; is it a pod process or a **system process** (kubelet, containerd, dockerd, journald, antivirus/security agents, log shippers)?

2. Read the CPU breakdown in `top`:

- `us` high -> user processes (app or agent).
- `sy` high -> kernel work: too many syscalls, context switching, network interrupts, iptables with thousands of rules (kube-proxy).
- `wa` high -> waiting on disk IO, not real compute.
- `st` high -> **steal time**, the hypervisor gives your CPU to other VMs, or a burstable T-instance ran out of CPU credits.
- `si` high -> softirq, heavy network packet processing.

3. Pods with **no CPU limits** or `BestEffort` pods may burst and consume the node, while still passing health checks.
4. Zombie/runaway processes, cron jobs, or leftover containers not managed by Kubernetes (`crictl ps -a`).
5. Kubelet itself busy: image garbage collection, too many pods, frequent probes, log rotation.
6. Security incident: crypto-miner - check unknown processes and outbound connections.
7. Compare `kubectl top pods` vs node `top` - the gap is the non-pod usage.

If one known service is at 100% CPU: find the PID (`top`, `pidstat`), then the hot thread (`top -H -p <PID>`); for Java run `jstack <PID>` and match the thread id in hex; use `strace -c -p <PID>` to see if it is spinning on syscalls, and `perf top -p <PID>` for hot functions. Check its logs and recent deploys/config changes (infinite retry loop, bad regex, GC thrash). Short-term: restart it or cap it (`systemctl set-property svc CPUQuota=50%`); long-term: fix the code and add CPU alerts.

Example:

```
kubectl top node ip-10-0-1-23
kubectl top pods -A --sort-by=cpu | head
# On the node:
top -o %CPU          # check us / sy / wa / st values
pidstat -u 2 5       # per-process CPU over time
crictl stats         # per-container CPU
mpstat -P ALL 1 3    # per-core; one core at 100% = single-threaded hog
```

> ***What to say to the interviewer:*** *I compare* `kubectl top` *with* `top` *on the node to see whether the load is from pods or from system components like kubelet, containerd or agents. Then I read the CPU breakdown - high system time, IO wait or steal time each point to different causes like iptables, disk bottlenecks or a noisy neighbour or exhausted T-instance credits - and I also check for pods without limits and unknown processes.*

Q46. Disk utilization on production servers reached 95%. What immediate and long-term actions would you take?

(also asked as: The / partition is full - how do you find and delete large files safely? / What is using up /var? / You got a low disk space alert - how do you clean up safely?)

> **In simple words:** Your cupboard is almost full. Right now you throw out the obvious junk so the door closes; later you buy a bigger cupboard, set a rule for old stuff, and put a sensor that warns you at 80%.

Detailed answer

Immediate (stop the bleeding, safely):

1. Find what is big: `df -h`, then `du -xh --max-depth=1 / | sort -h`, and `ncdu` if installed. Also `df -i` for inodes.
2. Check **deleted-but-open files**: `lsof +L1` - a deleted log still held by a process keeps using space until the process restarts or you truncate it.
3. Safe quick wins: `journalctl --vacuum-size=500M`, remove old rotated logs (`*.gz`), `apt clean`, old kernels (`apt autoremove`), `docker system prune` (careful on prod), old core dumps and temp exports.
4. Truncate a live huge log instead of deleting it: `truncate -s 0 app.log`.
5. Never delete database files or anything you do not understand - move/compress first.
6. If still critical: grow the volume online (EBS modify -> `growpart` -> `resize2fs`/`xfs_growfs`).

Long-term:

- Log rotation (`logrotate`, journald `SystemMaxUse`), ship logs to central storage (Loki/ELK/CloudWatch).
- Alert at 80% and 90% with trend-based "disk full in X hours" alerts.
- Separate volumes for `/var/log` or data; lifecycle cleanup cron; capacity planning.
- Write an RCA: what grew and why.

Example:

```
df -h; df -i
sudo du -xh --max-depth=1 /var | sort -h | tail
sudo lsof +L1 | awk '$7 > 100000000'        # deleted files still open, >100MB
sudo journalctl --vacuum-size=500M
sudo truncate -s 0 /var/log/myapp/app.log
# Grow EBS after modifying volume size in AWS:
sudo growpart /dev/nvme0n1 1 && sudo resize2fs /dev/nvme0n1p1
```

> ***What to say to the interviewer:*** *Immediately I find the biggest consumers with* `du`*, check for deleted-but-open files with* `lsof +L1`*, and free space safely by vacuuming journald, removing old rotated logs, and truncating huge live logs, or grow the EBS volume online if needed. Long term I fix log rotation, ship logs centrally, add 80/90 percent and trend alerts, and document the root cause.*

Q47. One of your EC2/Linux nodes keeps going out of memory. How would you diagnose and prevent it?

(also asked as: How do you locate and stop a process consuming high memory? / A server shows consistently high memory usage - what do you check?)

> **In simple words:** A room keeps overflowing with guests and the bouncer (OOM killer) keeps throwing someone out. You check the guest list to see who is bringing too many friends, then set a limit per guest or get a bigger room.

Detailed answer

Diagnose:

1. Confirm OOM kills: `dmesg -T | grep -i -E "killed process|out of memory"` or `journalctl -k | grep -i oom`. The log names the killed process and its memory.
2. Current usage: `free -h` (watch **available**), `ps aux --sort=-%mem | head`, `smem` for real per-process usage.
3. Trend over time: CloudWatch (needs the **CloudWatch agent** - memory is not a default EC2 metric) or Prometheus node-exporter. A steady climb = **memory leak**; sudden spikes = a batch job/traffic burst.
4. Check what changed: new deployment, config change (e.g. JVM `-Xmx`, PHP-FPM/Gunicorn worker counts too high), cache growth.
5. On Kubernetes nodes: pods without memory limits, too many pods, and missing `kube-reserved`/`system-reserved` so system daemons get starved.

Consistently high memory (no OOM kills): first check it is real. Linux fills spare RAM with page cache (`buff/cache` in `free -h`), which is released on demand, so only "available" dropping and swap in use (`vmstat si/so`) mean real pressure. Then trend it: flat = normal working set (maybe right-size); a steady climb = leak.

Locate and stop a memory hog (right now): `ps aux --sort=-%mem | head` or `top` then `M` -> note the PID and owner. If it is a managed service, restart it properly (`systemctl restart svc`) rather than killing it. Otherwise `kill <PID>` (SIGTERM, lets it clean up), wait, and use `kill -9` only if it ignores that. Capture evidence first if possible (heap dump, `pmap -x <PID>`), because a restart hides the leak.

Prevent:

- Right-size the instance or tune the app (worker counts, heap size).
- Set memory limits (systemd `MemoryMax=`, container/pod limits) so one process cannot eat the node.
- Add swap only as a buffer on non-latency-critical servers (not usually on K8s nodes).
- Alert on memory above 85% and on OOM events; fix leaks with the dev team using heap dumps.

Example:

```
dmesg -T | grep -i "killed process"
# [Tue Sep 22 03:14:07 2026] Out of memory: Killed process 2143 (java) total-vm:6.1GB, anon-rss:3.8G
    B
free -h
ps aux --sort=-%mem | head -5
# Cap a service with systemd
sudo systemctl set-property myapp.service MemoryMax=1500M
```

> ***What to say to the interviewer:*** *I confirm the OOM kill in* `dmesg` *or the kernel journal to see which process was killed, then check per-process memory and the trend over time with the CloudWatch agent or node-exporter to tell a leak from a spike. To prevent it, I right-size or tune the app's workers and heap, set memory limits via systemd or pod limits, reserve memory for system daemons on K8s nodes, and alert on high memory and OOM events.*

Q48. Latency spikes suddenly in production during off-peak hours. Where do you start debugging?

In simple words: If the roads are jammed at 3 AM when nobody is driving, it is not rush hour - it is probably road maintenance. Off-peak slowness usually comes from scheduled background work, not users.

Detailed answer

Low traffic + high latency points to **scheduled or internal** causes. Order of checks:

1. **Correlate the time**: does it happen at the same minute every day/hour? That strongly suggests cron jobs, backups, batch/ETL jobs, log rotation, DB maintenance (vacuum, index rebuild), snapshots, antivirus scans, or cache expiry.
2. **Scope it**: all endpoints or one? All hosts or one AZ/node? Use p95/p99 per endpoint and per instance.
3. **Scale-in side effects**: autoscaling removed capacity at night, so the few remaining instances or cold caches struggle; scaled-to-zero services have **cold starts**; JIT/connection pools re-warm.
4. **Host level** at that minute: `sar`/`iostat` for IO wait (backups hammering disk), CPU steal or exhausted burst credits on T-instances, GC pauses in logs.
5. **Network/DNS**: DNS TTL expiry, connection pools idle-timed out by NAT/LB (350s NAT idle timeout on AWS) -> new handshakes; certificate or token refresh.
6. **Dependencies**: DB slow query log, third-party API maintenance windows.
7. Use traces to see which span grows.

Example:

```
crontab -l; ls /etc/cron.d/; systemctl list-timers        # scheduled jobs
sar -u -d -f /var/log/sysstat/sa22 -s 02:50:00 -e 03:20:00  # CPU/disk at spike time
grep -i "pause" /var/log/myapp/gc.log | tail
curl -o /dev/null -s -w "connect:%{time_connect} ttfb:%{time_starttransfer}\n" https://api/x
```

What to say to the interviewer: *Because traffic is low, I suspect internal causes first, so I correlate the spike time with cron jobs, backups, batch jobs, DB maintenance and autoscaling scale-in or cold starts. Then I scope it by endpoint and host using p99 metrics and traces, and check host IO wait, CPU steal or credits, GC pauses, and idle connection timeouts at that exact window.*

Q49. A user cannot log in to a Linux server. How do you troubleshoot?

In simple words: Someone cannot enter the office. Is their ID card expired, blocked, the wrong card, is the door itself broken, or is their desk (home folder) missing?

Detailed answer

1. Get the exact error and method (SSH key, password, console).
2. Read the auth log: `/var/log/auth.log` (Ubuntu) or `/var/log/secure` (RHEL), or `journalctl -u sshd`.
3. Account state: `id user`, `chage -l user` (password expired?), `passwd -S user` (L = locked), `faillock --user user` or `pam_tally2` (locked after failed attempts).

4. Shell: `/etc/passwd` shell set to `/sbin/nologin` or `/bin/false`?
5. SSH rules: `AllowUsers`/`AllowGroups`/`DenyUsers` in `/etc/ssh/sshd_config`; key in `~/.ssh/authorized_keys` with correct permissions.
6. Home directory missing or wrong owner.
7. Disk full (`df -h`) or too many processes can also block logins.

Example:

```
sudo tail -f /var/log/auth.log
passwd -S priya;  chage -l priya
sudo faillock --user priya --reset
getent passwd priya          # check shell and home
sudo usermod -s /bin/bash priya
```

> ***What to say to the interviewer:*** *I read the auth log first because it usually names the reason, then check whether the account is locked or expired with* `passwd -S`*,* `chage -l` *and faillock, whether the shell is nologin, and sshd AllowUsers rules and key permissions. I also check the home directory and disk space.*

Q50. A script is executable by one user but not another, or a user cannot access a directory even though they are in the correct group. Why?

(also asked as: A user cannot access a directory even though they are in the correct group)

> **In simple words:** Two people have the same door key, but one is not on the building's allowed list, or the corridor leading to the door is locked for them.

Detailed answer

Check in this order:

- File permissions: `ls -l script.sh` -> maybe `rwxr-x---`, so "others" cannot execute. The second user is not the owner or in the group.
- Parent directories: every directory in the path needs `x` for that user (`namei -l /opt/scripts/script.sh` shows each level).
- Interpreter: the shebang (`#!/usr/bin/env bash`) must be executable by them.
- Mount options: a filesystem mounted `noexec` (like `/tmp` on hardened servers) blocks execution for everyone running it from there.
- ACLs: `getfacl script.sh` may explicitly deny a user.
- SELinux/AppArmor contexts: `ls -Z`.
- Inside the script: it may read files or use sudo that only user A is allowed.
- `PATH` differences: user B runs a different copy.
- Group just added? Group membership is read at login, so the user's current session still has the OLD groups - `id` in their shell will not show it. Log out and in again, or run `newgrp devops` for that shell.
- Directory needs the right bits: `r` to list, `x` to enter, `w+x` to create or delete files inside.
- NFS/EFS shares: the server checks numeric UIDs/GIDs, so a GID mismatch between servers or `root_squash` blocks access.

- Immutable attribute: `lsattr` shows `i`, and then even root cannot modify it (`chattr -i` to remove).

Example:

```
ls -l /opt/scripts/deploy.sh      # -rwxr-x--- root devops
namei -l /opt/scripts/deploy.sh
getfacl /opt/scripts/deploy.sh
id userb                          # not in devops group
sudo usermod -aG devops userb     # fix (user must log in again)
```

> ***What to say to the interviewer:*** *I compare the file's owner, group and mode with the second user's groups using* `ls -l` *and* `id`*, check every parent directory with* `namei -l`*, and look at ACLs, noexec mounts and SELinux contexts. Usually the fix is adding the user to the right group or adjusting group permissions, not chmod 777.*

Q51. You cannot SSH into a server. How do you troubleshoot?

(also asked as: SSH is not working)

> **In simple words:** You cannot enter a house. Either the road to it is blocked (network), nobody opens the door (sshd down), or the door opens but your key does not fit (authentication).

Detailed answer

1. Read the error: `Connection timed out` -> network/firewall; `Connection refused` -> host reachable, sshd not listening; `Permission denied (publickey)` -> auth problem; `Host key verification failed` -> host key changed (rebuilt server).
2. Verbose client: `ssh -vvv user@host` shows the exact stage that fails.
3. Network: `nc -zv host 22`, Security Group/NACL inbound 22 from your IP, route tables, public IP or bastion path.
4. Auth: right user (`ec2-user` vs `ubuntu`), right key, key file mode `600`, server `~/.ssh` 700 and `authorized_keys` 600, correct owner.
5. Server side, via another path (EC2 **Serial Console**, **SSM Session Manager**, console): `systemctl status sshd`, `sshd -t` (config syntax), `/var/log/auth.log`, disk full, `fail2ban` ban.
6. Last resort on AWS: stop instance, attach the root volume to a rescue instance, fix config/keys.

Example:

```
ssh -vvv -i key.pem ec2-user@10.0.1.5
nc -zv 10.0.1.5 22
chmod 600 key.pem
aws ssm start-session --target i-0abc123     # no SSH needed
sudo sshd -t && sudo systemctl restart sshd
```

> ***What to say to the interviewer:*** *I read the exact error with* `ssh -vvv` *to classify it as network, sshd or authentication. Then I check port 22 and security groups for timeouts, sshd status and config for refusals, and user, key and permissions for auth failures, using SSM Session Manager or the serial console to get in if SSH is fully broken.*

Q52. Ping to a server works, but SSH using its hostname fails. Why?

In simple words: You can reach the house by its exact GPS coordinates, but using its name sends you to a different house, or the house is open for waving (ping) but its door (port 22) is locked.

Detailed answer

- Did ping use the **IP** and SSH the **name**? Then it is DNS: `dig host`, `getent hosts host` - the name may resolve to a different/old IP, IPv6 (AAAA) first, or a wrong `/etc/hosts` entry.
- If ping by name also works: ping is ICMP, SSH is TCP 22 - the firewall/Security Group may allow ICMP but not port 22. Test `nc -zv host 22`.
- `~/.ssh/config` may override the hostname, user, port or ProxyJump for that name.
- `known_hosts` mismatch for that name ("REMOTE HOST IDENTIFICATION HAS CHANGED") if the name now points to a rebuilt server.
- sshd running on a non-default port.
- Slow login rather than failure: `UseDNS yes` on the server doing reverse lookups.

Example:

```
getent hosts app01.internal; dig +short app01.internal
ssh -G app01.internal | grep -E "^(hostname|port|user) "
nc -zv app01.internal 22
ssh-keygen -R app01.internal         # remove stale host key
```

What to say to the interviewer: *Ping only proves ICMP to some IP works, so I check whether the hostname resolves to the same IP with* `getent hosts` *or dig, whether port 22 is actually open with* `nc -zv`*, and whether ~/.ssh/config or a stale known_hosts entry is interfering.*

Q53. A log file shows junk characters when you open it. What could be wrong?

In simple words: It is like opening a zipped parcel and trying to read the packing foam - the content is fine, you are just looking at it in the wrong form.

Detailed answer

- It is **compressed**: rotated logs like `app.log.1.gz` -> use `zcat`, `zless`, `zgrep`. Check with `file app.log`.
- It is **binary**: systemd journal files, `wtmp`/`btmp` (use `last`, `lastb`), or database files.
- **Encoding** mismatch: UTF-16 from a Windows app, or a different charset -> `file -i`, `iconv -f UTF-16 -t UTF-8`.
- **ANSI color codes** from apps (`^[[32m`) -> `less -R` or strip with sed.
- **NUL bytes** at the start after log rotation with `copytruncate` while the app keeps writing at its old offset -> the file becomes "sparse" with zeros. Fix: make the app open with append mode or reopen logs on rotation (signal/postrotate) instead of copytruncate.
- Disk/filesystem corruption (rare) -> check `dmesg`.

Example:

```
file /var/log/app.log.2.gz          # gzip compressed data
zgrep "ERROR" /var/log/app.log.2.gz
file -i app.log                     # charset=utf-16le
iconv -f UTF-16LE -t UTF-8 app.log > app-utf8.log
less -R colored.log
tr -d '\000' < app.log | less       # strip NUL bytes
```

What to say to the interviewer: *I run* `file` *on it first: it is often a gzip-rotated log to read with zcat or zgrep, a binary file like journal or wtmp, a UTF-16 encoding to convert with iconv, or ANSI colour codes. NUL bytes at the start usually mean logrotate copytruncate while the app keeps its old offset.*

Q54. A cron job is not running. How do you debug it?

(also asked as: A cron job did not run last night - what do you check?)

In simple words: The alarm clock did not ring. Is the clock plugged in (cron service), was the time set right, and when it rang did the person actually know how to do the task without their usual tools (environment)?

Detailed answer

1. Is cron running? `systemctl status cron` (Ubuntu) / `crond` (RHEL).
2. Did it trigger? `grep CRON /var/log/syslog` or `journalctl -u cron` -> if the line is there, cron ran it and the **script** failed.
3. Syntax: check the 5 time fields (crontab.guru logic), and in `/etc/crontab` or `/etc/cron.d/` there is an extra **user** field. Files in `/etc/cron.d` must not have dots in the name and need a trailing newline.
4. Environment: cron has minimal `PATH` and no profile -> use absolute paths, set `PATH=` at the top, `cd` to the right directory.
5. `%` is special in crontab (means newline) -> escape as `\%` (common with `date +%F`).
6. Permissions: script executable? `/etc/cron.allow` / `cron.deny`? Correct user's crontab?
7. Capture output: `>> /tmp/job.log 2>&1` and test with `env -i /bin/sh -c '/path/script.sh'` to mimic cron.
8. Timezone of the server vs what you expected (cron uses system time).

Example:

```
systemctl status cron
grep CRON /var/log/syslog | tail
crontab -l
# wrong:  0 2 * * * backup.sh
# right:
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
0 3 * * * /usr/bin/tar czf /backup/app-$(date +\%F).tgz /opt/app
```

What to say to the interviewer: *I confirm the cron service runs and check syslog or journalctl to see whether the job was triggered; if it was, the script is failing, usually because of cron's minimal PATH, relative paths, or an unescaped % sign. I redirect output to a log file and reproduce with* `env -i` *to mimic cron's environment.*

Q55. yum/dnf or apt is failing. How do you troubleshoot it?

In simple words: The shop you order parts from is not delivering. Maybe you cannot reach the shop (network/proxy), the catalogue is outdated (metadata), the shop's ID is not trusted (GPG key), or another order is still being processed (lock).

Detailed answer

1. Read the full error - it usually names the repo or package.
2. Network: can you reach the repo? `curl -I <repo-url>`, DNS, proxy settings, NAT gateway / VPC endpoint for private instances.
3. Lock: another apt/dnf or `unattended-upgrades` is running -> wait; `ps aux | grep -E "apt|dpkg"`. Only remove stale lock files if no process holds them.
4. Stale metadata: `apt clean && apt update` / `dnf clean all && dnf makecache`.
5. GPG/key errors: import the repo key (expired keys are common with third-party repos).
6. Broken or interrupted installs: `dpkg --configure -a`, `apt --fix-broken install`; `dnf check`, `package-cleanup --problems`.
7. Bad or dead third-party repo: disable it (`/etc/apt/sources.list.d/`, `dnf config-manager --set-disabled`).
8. Disk full on `/var` or `/boot`; clock wrong (TLS/GPG validity errors).

Example:

```
sudo apt update 2>&1 | tail
sudo lsof /var/lib/dpkg/lock-frontend
sudo dpkg --configure -a && sudo apt --fix-broken install
sudo dnf clean all && sudo dnf makecache
curl -I https://archive.ubuntu.com/ubuntu/
df -h /var /boot; timedatectl
```

What to say to the interviewer: *I read the exact error, then check repo reachability, DNS and proxy, whether another package process holds the lock, and refresh metadata with clean and update. Other common causes are expired GPG keys, dead third-party repos, interrupted installs needing* `dpkg --configure -a`*, a full /var or /boot, and a wrong system clock.*

Q56. A user is in the sudoers file but sudo still fails. Why?

In simple words: Your name is on the VIP list, but the list has a typo, a later line overrides you, or the bouncer is checking a different list.

Detailed answer

- Syntax error in `/etc/sudoers` or a file in `/etc/sudoers.d/` -> sudo may refuse to work at all. Always edit with `visudo` and check with `visudo -c`.
- Rule order: sudoers uses the **last matching rule** - a later `%group` line may restrict the user.
- Group membership added but the user did not log out/in -> `id` in their session will not show the new group.
- Rule limited to certain commands or hosts, or `!` negations.
- Files in `/etc/sudoers.d/` ignored if their name contains a `.` or `~`, or have wrong permissions (should be `0440`, owned by root).

- Password expired, account locked, or `requiretty` settings.
- Messages: check `/var/log/auth.log` or `/var/log/secure` - "user NOT in sudoers" vs "command not allowed".

Example:

```
sudo visudo -c
sudo -l -U priya              # what is this user actually allowed?
id priya
ls -l /etc/sudoers.d/         # -r--r----- root root  90-priya
echo 'priya ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx' | sudo tee /etc/sudoers.d/90-priya
sudo chmod 440 /etc/sudoers.d/90-priya
```

What to say to the interviewer: *I check syntax with* `visudo -c`*, see the user's effective rules with* `sudo -l -U user`*, and confirm with* `id` *that a newly added group is active in their session. Common causes are a later rule overriding theirs, a sudoers.d file with a dot in its name or wrong permissions, and command-restricted rules; the auth log shows which one.*

Q57. A server hit a kernel panic. How do you troubleshoot and recover?

In simple words: The building's power system tripped completely. First get power back (reboot / older kernel), then read the black box recorder (crash dump, logs) to see what caused the trip.

Detailed answer

Recover:

1. Reboot. If it panics again, pick an **older kernel** in GRUB or recovery mode (on AWS use the EC2 Serial Console, or attach the root volume to a rescue instance).
2. Common triggers: new kernel or driver/module after patching, bad fstab or missing root device, corrupted initramfs, hardware/memory faults, out-of-memory with `panic_on_oom`.
3. Fixes: boot the old kernel and remove/hold the bad one, rebuild initramfs (`dracut -f` / `update-initramfs -u`), fix fstab, fsck the root disk.

Find the root cause:

- Screenshot/serial console output (EC2 "Get system log" / "Get instance screenshot").
- `journalctl -k -b -1` (previous boot kernel log) if persistent journald is enabled.
- **kdump** captures a crash dump in `/var/crash`, analysed with the `crash` tool.
- Recent changes: new kernel, modules, hardware events.

Example:

```
aws ec2 get-console-output --instance-id i-0abc123 --latest --output text | tail -40
journalctl -k -b -1 | tail -50
ls /var/crash/
sudo dracut -f            # RHEL: rebuild initramfs
sudo update-initramfs -u  # Ubuntu
sudo systemctl enable --now kdump
```

What to say to the interviewer: *I bring it back first, booting the previous kernel or using recovery mode or a rescue instance, then find the cause from the console output, the previous boot's kernel log and a kdump crash dump. The usual culprits are a new kernel or driver, a broken initramfs or fstab, or hardware, and I enable kdump so the next panic leaves evidence.*

Q58. How do you find and handle zombie processes?

In simple words: A zombie is a worker who has finished and gone home, but their name is still on the attendance sheet because the manager (parent process) never signed them out.

Detailed answer

- A zombie (Z state, `<defunct>`) has already exited; it uses no CPU or memory, only a PID table entry. It exists because the **parent** did not call `wait()` to collect its exit status.
- You **cannot kill a zombie** - it is already dead. `kill -9` on it does nothing.
- Fix the parent: send it `SIGCHLD` (`kill -s SIGCHLD <PPID>`) to prompt it to reap; if that fails, restart/kill the parent -> the zombies are adopted by init/systemd (PID 1), which reaps them.
- A few zombies are harmless; thousands can exhaust the PID limit (`pid_max`) and stop new processes starting.
- In containers, if your app is PID 1 and does not reap children, use `tini` or `docker run --init` (or `shareProcessNamespace` patterns) - a common cause.
- Long-term: fix the parent's code to reap children.

Example:

```
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/'
#  4521  4400  Z   [worker] <defunct>
ps -o pid,cmd -p 4400                 # the parent
sudo kill -s SIGCHLD 4400
sudo systemctl restart myapp          # if the parent will not reap
```

What to say to the interviewer: *I list them with* `ps` *filtering for Z state and find the parent PID, because a zombie is already dead and cannot be killed; the parent has not reaped it. I signal the parent with SIGCHLD or restart it so PID 1 adopts and reaps them, and in containers I use tini or* `--init` *as PID 1.*

Q59. An application fails with "Too many open files". How do you fix it?

In simple words: Every process has a limited number of drawer handles it can hold at once. The app is holding too many - either it needs a bigger allowance or it is forgetting to let go (leak).

Detailed answer

1. Confirm the limit and usage: `cat /proc/<PID>/limits | grep "open files"` and `ls /proc/<PID>/fd | wc -l` (or `lsof -p PID | wc -l`).
2. Leak or real need? If the count keeps rising with steady traffic, it is a **file/socket leak** (connections not closed). Look at `lsof -p PID` - many sockets in `CLOSE_WAIT` point to the app not closing connections.
3. Raise the limit correctly:

- systemd services: `LimitNOFILE=65536` in the unit (or a drop-in), then `daemon-reload` and restart. (`/etc/security/limits.conf` does NOT apply to systemd services.)
- Login sessions: `/etc/security/limits.conf` (`nofile`), then re-login; check with `ulimit -n`.
- System-wide max: `fs.file-max` / `fs.nr_open` via sysctl (rarely the actual problem).

- Containers: `--ulimit nofile=65536:65536`.

4. Monitor fd usage so it alerts before the limit.

Example:

```
PID=$(pgrep -f myapp | head -1)
grep "open files" /proc/$PID/limits      # Max open files 1024 1024
ls /proc/$PID/fd | wc -l                 # 1021
lsof -p $PID | awk '{print $NF}' | sort | uniq -c | sort -rn | head
sudo systemctl edit myapp                # add: [Service] LimitNOFILE=65536
sudo systemctl daemon-reload && sudo systemctl restart myapp
```

> ***What to say to the interviewer:*** *I compare the process's current descriptor count in /proc/PID/fd with its limit in /proc/PID/limits and check with lsof whether it is a leak, such as sockets piling up in CLOSE_WAIT. If the app genuinely needs more, I raise LimitNOFILE in the systemd unit, since limits.conf does not apply to services, and add monitoring on descriptor usage.*

Q60. A user's home directory is missing. How do you restore it?

> **In simple words:** An employee's locker was removed. You give them a new empty locker with the standard starter kit, and if their old belongings matter you fetch them from the storage backup.

Detailed answer

1. Confirm: `getent passwd user` shows the expected home path; check whether it was moved or is on a mount that did not come up (NFS/EFS home, `/home` on a separate disk -> `df -h`, `mount -a`).
2. Recreate a fresh one: `mkhomedir_helper user` (or `mkdir` + copy `/etc/skel`), set owner `user:user` and mode `700`/`750`.
3. Restore data from backup/snapshot (EBS snapshot -> new volume -> mount and copy with `rsync -a` to keep permissions).
4. Restore `~/.ssh/authorized_keys` so SSH key login works again (700 on `.ssh`, 600 on the file).
5. If it happens for many users, enable `pam_mkhomedir` so homes are auto-created on first login (common with LDAP/AD users).
6. Investigate why: `userdel -r`, cleanup script, failed mount - check auth logs and bash history.

Example:

```
getent passwd priya            # priya:x:1001:1001::/home/priya:/bin/bash
sudo mkhomedir_helper priya    # or:
sudo cp -r /etc/skel /home/priya && sudo chown -R priya:priya /home/priya && sudo chmod 750 /home/pr
    iya
sudo rsync -a /mnt/snapshot/home/priya/ /home/priya/
```

> ***What to say to the interviewer:*** *I first check whether it is truly deleted or just on a mount that failed, then recreate it with mkhomedir_helper or /etc/skel with the right ownership and permissions, and restore data and SSH keys from a backup or snapshot using rsync -a. For directory-based users I enable pam_mkhomedir, and I find out what removed it.*

Q61. The server time is wrong. How do you fix it using NTP/chrony?

In simple words: If a server's watch is wrong, it arrives late to every meeting - certificates look expired, logs do not line up, and logins with time-based tokens fail. NTP keeps checking the watch against an atomic clock.

Detailed answer

1. Check: `timedatectl` -> "System clock synchronized: no" / "NTP service: inactive"? Also check the **timezone** - maybe the time is right but the zone is wrong.
2. Use **chrony** (default on RHEL/Amazon Linux, common on Ubuntu) or systemd-timesyncd.
3. Configure servers in `/etc/chrony.conf`: on AWS use the **Amazon Time Sync Service** `169.254.169.123` (no internet needed); otherwise `pool.ntp.org` or company NTP.
4. Make sure UDP 123 outbound is allowed if using external servers.
5. Large offsets: chrony slews (slowly adjusts) by default; `chronyc makestep` jumps immediately (careful with running databases/apps).
6. Verify with `chronyc tracking` and `chronyc sources -v`.
7. Monitor clock offset (node-exporter `node_timex_offset_seconds`).

Example:

```
timedatectl
sudo timedatectl set-timezone Asia/Kolkata
echo "server 169.254.169.123 prefer iburst" | sudo tee -a /etc/chrony.conf
sudo systemctl enable --now chronyd
sudo chronyc makestep
chronyc tracking | grep -E "System time|Leap"
chronyc sources -v
```

What to say to the interviewer: *I check* `timedatectl` *for sync status and timezone, configure chrony with a reliable source - on AWS the Amazon Time Sync Service at 169.254.169.123 - and restart it, using* `chronyc makestep` *for a large offset. I verify with* `chronyc tracking` *and monitor clock drift, because wrong time breaks TLS, tokens, clusters and log correlation.*

Q62. You deleted large files but disk space was not freed. Why? What happens when a file is deleted while still in use?

(also asked as: What happens when a file is deleted but still in use?)

In simple words: Deleting a file removes its name from the index, but if someone is still reading the book, the library cannot recycle the paper until they give it back.

Detailed answer

- On Linux, `rm` removes the **directory entry** (the name/link). The data blocks and inode are freed only when the link count is 0 **and** no process has the file open.
- A running process (e.g. an app writing to its log) keeps writing to the deleted file, invisible to `ls` and `du` but still using space shown by `df`.
- Find them: `lsof +L1` or `lsof | grep deleted`.
- Fix: restart/reload the process (logrotate `postrotate` does this), or truncate through its file descriptor without restart: `: > /proc/<PID>/fd/<FD>`.

- Prevention: truncate live logs (`truncate -s 0`) instead of `rm`, and use proper logrotate.
- Bonus: while the process runs, the deleted file can still be **recovered** by copying `/proc/<PID>/fd/<FD>`.

Example:

```
sudo lsof +L1
# COMMAND  PID USER  FD   TYPE SIZE/OFF    NLINK NAME
# java    2143 app   12w  REG  8589934592  0     /var/log/app.log (deleted)
sudo sh -c ': > /proc/2143/fd/12'     # free space without restart
sudo systemctl restart myapp           # or restart the process
```

> ***What to say to the interviewer:*** *Deleting a file only removes its name; the space is freed when no process holds it open, so an app still writing to a deleted log keeps consuming space that df shows but du does not. I find those with `lsof +L1` and either restart the process or truncate it through /proc/PID/fd, and prevent it by truncating instead of deleting live logs.*

Q63. df and du show different disk usage. Why?

> **In simple words:** df reads the building's official meter, du walks around counting boxes it can see. If some boxes are hidden or thrown out but still held by someone, the two numbers disagree.

Detailed answer

Main reasons, most common first:

1. **Deleted-but-open files** -> counted by `df`, invisible to `du`. Check `lsof +L1` (see previous question).
2. **Files hidden under a mount point** -> data written to `/data` before a disk was mounted on `/data` sits underneath, hidden. Check with a bind mount: `mount --bind / /mnt/root && du -sh /mnt/root/data`.
3. **Reserved blocks** -> ext4 reserves ~5% for root, so df "Used + Avail" is less than "Size". Can reduce with `tune2fs -m 1` on data disks.
4. Filesystem metadata/journal overhead.
5. `du` run without `-x`, or without permission to read some folders, or double-counting across mounts.
6. Sparse files and hard links count differently (`du --apparent-size`).

Example:

```
df -h /; sudo du -xsh /
sudo lsof +L1 | head
sudo mkdir -p /mnt/rootfs && sudo mount --bind / /mnt/rootfs
sudo du -sh /mnt/rootfs/data           # data hidden under the /data mount?
sudo umount /mnt/rootfs
sudo tune2fs -l /dev/nvme0n1p1 | grep "Reserved block count"
```

> ***What to say to the interviewer:*** *df reads filesystem-level usage while du sums the files it can see, so the gap is usually deleted files still held open, which I find with `lsof +L1`, or data hidden underneath a mount point, which I reveal with a bind mount. Reserved blocks on ext4 and metadata overhead explain smaller differences.*

Q64. The system load is very high. How do you analyse it?

In simple words: Load average is the length of the queue at the counters. A long queue can mean the cashiers are all busy (CPU) or everyone is waiting for the stockroom (disk IO) - the fix is different.

Detailed answer

1. `uptime` -> 1, 5, 15-minute load averages. Compare to CPU count (`nproc`): load 8 on 8 cores is full, on 2 cores it is overloaded. Rising 1-min vs 15-min = getting worse.
2. On Linux load = runnable processes (**R**) + processes in uninterruptible sleep (**D**, usually IO). So high load does not always mean high CPU.
3. `top` / `vmstat 1`:

- High `us`/`sy` and many in `r` column -> CPU-bound -> find the process (`ps --sort=-%cpu`, `pidstat`).
- High `wa` and many in `b` column / `D` state -> IO-bound -> `iostat -x`, `iotop` (slow disk, EBS limits, NFS hang).

4. Hung NFS mounts can put many processes in `D` state and send load through the roof with idle CPU.
5. Memory pressure -> swapping (`si`/`so` in vmstat) also raises load.
6. Correlate with time (cron, backups, deploys) and fix: scale, tune, move the job, or fix the storage.

Example:

```
uptime; nproc
# load average: 24.10, 18.40, 9.02    (4 cores -> overloaded, getting worse)
vmstat 1 5          # r, b, wa, si/so columns
ps -eo stat,pid,cmd | awk '$1 ~ /^D/'   # stuck in IO wait
iostat -xz 1 3
```

What to say to the interviewer: *I compare the load average with the number of cores and remember Linux load counts both runnable and IO-blocked processes. Then vmstat and top tell me whether it is CPU-bound, IO-bound, a hung NFS mount with processes in D state, or swapping, and I drill down with pidstat, iostat or iotop and correlate it with scheduled jobs.*

Q65. An NFS mount is not working. How do you troubleshoot it?

In simple words: You are trying to use a shared cupboard in another building. The road may be closed (network/ports), the other building may not have given you permission (exports), or your door key type does not match (NFS version).

Detailed answer

1. Error message: `mount.nfs: access denied`, `timed out`, `No such file or directory`, or a hang.
2. Network: `ping`/`nc -zv server 2049` (NFSv4 only needs TCP 2049; v3 also needs 111 rpcbind and mountd ports). On AWS EFS: the mount target's Security Group must allow **2049 from the client**.
3. Server exports: `showmount -e server` (v3) and `/etc/exports` - is the client IP/subnet allowed? Run `exportfs -ra` after changes.

4. Client packages: `nfs-utils` (RHEL) / `nfs-common` (Ubuntu); `amazon-efs-utils` for EFS.
5. Version mismatch: force `-o vers=4.1`.
6. Permission issues after mounting: UID/GID mismatch between client and server, `root_squash`.
7. Hung (stale) mount: `umount -f -l /mnt/nfs`; use `_netdev,nofail` in fstab so boot does not hang.
8. Logs: `dmesg`, `journalctl` on client; `/var/log/messages` on server.

Example:

```
nc -zv 10.0.3.10 2049
showmount -e 10.0.3.10
sudo mount -t nfs4 -o vers=4.1,timeo=600,retrans=2 10.0.3.10:/exports/data /mnt/data
dmesg | tail
# /etc/fstab
10.0.3.10:/exports/data  /mnt/data  nfs4  vers=4.1,_netdev,nofail  0 0
```

> ***What to say to the interviewer:*** *I read the mount error and test port 2049 - on EFS that means the mount target security group - then check the server's exports allow my client and that the client has nfs-utils. After that I look at NFS version mismatches, UID/GID and root_squash permission issues, and use* `_netdev,nofail` *in fstab so a failed share does not block boot.*

Q66. How do you recover a deleted file on Linux?

(also asked as: An important config file was deleted and there is no backup - what do you do?)

> **In simple words:** Linux has no recycle bin. You can grab the file back if someone is still holding it, fetch a copy from the backup cupboard, or try forensic tools that search the floor for the scraps - which get swept away quickly.

Detailed answer

In order of success rate:

1. **Still open by a process?** `lsof +L1 | grep filename` -> copy it back from the file descriptor: `cp /proc/<PID>/fd/<FD> /restore/file`. Works perfectly and instantly.
2. **Backups/snapshots**: EBS snapshot -> create volume -> mount -> copy; AWS Backup; S3 versioning; LVM snapshots; git history for config/code.
3. **Undelete tools** (last resort): `extundelete` / `ext4magic` for ext4, `xfs_undelete` for XFS, `testdisk`/`photorec` for raw carving. Immediately **stop writing** to that filesystem - remount read-only or unmount - because new writes overwrite freed blocks. Recover to a DIFFERENT disk.
4. Prevention: backups with tested restores, `alias rm='rm -i'` for humans, trash tools, and config under version control.

Deleted config file with no backup:

- If the service is still running, its config is often still loaded in memory - do NOT restart it yet. Check `/proc/<PID>/fd` in case the file is open, and dump the running config if the app supports it (`nginx -T`, `haproxy -c`, database `SHOW VARIABLES`).

- Package default: find the owning package (`dpkg -S /etc/nginx/nginx.conf`, `rpm -qf`) and reinstall it - `apt-get install --reinstall nginx` (add `-o Dpkg::Options::="--force-confmiss"` so a missing conffile is recreated) or `dnf reinstall nginx`. `rpm -V nginx` shows which files are missing or changed. Then re-apply your customisations.
- Other copies: the same file on a sibling server, `.bak`/`.rpmsave`/`.dpkg-old` files, the AMI/snapshot, **etckeeper** (git history of /etc).
- Real fix: keep configs in **config management** (Ansible/Git), so re-running the playbook IS the restore.

Example:

```
sudo lsof +L1 | grep report.csv
# python 3310 app 5r REG ... /data/report.csv (deleted)
sudo cp /proc/3310/fd/5 /tmp/report.csv
# Last resort, filesystem unmounted:
sudo umount /data
sudo extundelete /dev/nvme1n1p1 --restore-file data/report.csv --output-dir /recovery
```

> ***What to say to the interviewer:*** *If a process still has it open I copy it back from /proc/PID/fd immediately; otherwise I restore from a snapshot or backup. As a last resort I unmount or remount the filesystem read-only to avoid overwriting blocks and try extundelete or testdisk onto a different disk, and prevention is backups with regularly tested restores.*

Q67. A filesystem went read-only. How do you remount it read-write without a reboot?

> **In simple words:** The notebook locked itself to "view only" because it noticed a torn page. You can unlock it, but you should first check why it locked, or you may tear more pages.

Detailed answer

- The kernel remounts a filesystem read-only when it detects **errors** (`errors=remount-ro` in fstab) - disk/EBS I/O errors or corruption. So first read `dmesg` for EXT4-fs/XFS errors.
- If the cause is resolved (e.g. temporary storage glitch, or it was mounted ro on purpose/in recovery mode):
- `mount -o remount,rw /mountpoint` -> switch back to read-write without unmounting.
- If there are real filesystem errors, remount-rw is risky -> schedule an fsck (see next question) or fail over.
- Also check the disk is not simply full, which gives "No space left" rather than read-only.
- In recovery mode/single-user the root is mounted ro by default -> `mount -o remount,rw /` is the normal first step.

Example:

```
dmesg -T | grep -i -E "ext4-fs error|xfs|remount|i/o error" | tail
mount | grep " / "            # (ro,relatime,...)
sudo mount -o remount,rw /
touch /tmp/rwtest && echo OK
```

What to say to the interviewer: *I use* `mount -o remount,rw /mountpoint`*, but only after checking dmesg for why the kernel made it read-only, because usually it detected I/O errors or corruption. If there are real errors I plan an fsck or failover instead of forcing it writable.*

Q68. How do you fix a corrupted filesystem with fsck?

In simple words: fsck is a doctor that checks the filing system's index. You never operate on a patient who is running around - the filesystem must be unmounted (resting) first.

Detailed answer

- **Never run fsck on a mounted read-write filesystem** - it can cause more corruption.
- Snapshot the volume first (AWS: EBS snapshot) so the repair can be undone.
- Data disk: stop apps, `umount /data`, then `fsck -f /dev/xxx` (ext4 `e2fsck -fy`) or for XFS `xfs_repair /dev/xxx` (XFS does not use fsck; if the log is dirty, mount/unmount first to replay it, use `xfs_repair -L` only as a last resort because it discards the log).
- Root filesystem: boot into recovery/rescue mode, a live USB, or on AWS attach the root volume to a **rescue instance** and repair it there. Or force a check on next boot (`fsck.mode=force` kernel parameter, or `touch /forcefsck` on older systems).
- Check `lost+found` afterwards for recovered fragments.
- Find the cause: failing disk (SMART, EBS status checks), unclean shutdowns.

Example:

```
sudo umount /data
sudo e2fsck -fy /dev/nvme1n1p1     # ext4
sudo xfs_repair /dev/nvme1n1p1     # XFS
sudo mount /data && ls /data/lost+found
```

What to say to the interviewer: *I take a snapshot, unmount the filesystem - or for root use rescue mode or attach the volume to a rescue instance - and run* `e2fsck -f` *for ext4 or* `xfs_repair` *for XFS, never on a mounted filesystem. Afterwards I check lost+found and investigate the underlying disk or shutdown issue.*

Q69. The disk shows free space but you get "No space left on device". Why? (inode exhaustion)

In simple words: A car park with lots of empty space but no parking tickets left. Each file needs a ticket (inode); millions of tiny files use up the tickets before the space.

Detailed answer

- Every file uses one **inode**; ext4 fixes the inode count at format time. Millions of tiny files (sessions, cache, mail queue, temp files, small log fragments, container layers) can exhaust inodes while `df -h` shows free GB.
- Confirm: `df -i` -> `IUse%` at 100%.
- Find the folder with most files: `du --inodes -x / | sort -n | tail` (or `find /var -xdev -type f | cut -d/ -f2-3 | sort | uniq -c | sort -n | tail`).

- Fix: delete the tiny files (e.g. old PHP sessions, cache, `/var/spool`), using `find -delete` - `rm *` fails with "argument list too long".
- Long-term: fix the app's cleanup, add cron cleanup, alert on inode usage, or format data disks with more inodes (`mkfs.ext4 -i 4096`) or use XFS, which allocates inodes dynamically.

Example:

```
df -i /
# Filesystem     Inodes  IUsed IFree IUse% Mounted on
# /dev/nvme0n1p1 1310720 1310720   0  100% /
sudo du --inodes -x / 2>/dev/null | sort -n | tail -5
sudo find /var/app/cache -type f -mtime +2 -delete
```

> ***What to say to the interviewer:*** *That is usually inode exhaustion, which I confirm with* `df -i`*; millions of tiny files use up all inodes while there is still free space. I find the directory with the most files using* `du --inodes`*, clean them up with* `find -delete`*, then fix the app's cleanup, alert on inode usage, or use XFS or more inodes for that disk.*

Q70. Apache is running, but the website is not accessible. How do you troubleshoot it?

> **In simple words:** The shop is open and the lights are on, but customers cannot get in. Maybe the front door is on the wrong street (port/interface), a guard blocks the entrance (firewall/SELinux), the signboard points elsewhere (DNS/vhost), or the shelves are locked (file permissions).

Detailed answer

Work from the inside out:

1. Is it really healthy? `systemctl status httpd` (RHEL) / `apache2` (Ubuntu), and `apachectl configtest` for syntax.
2. Is it listening where you think? `ss -tlnp | grep -E ':80|:443'` -> `127.0.0.1:80` means local only; check `Listen` directives.
3. Test locally: `curl -I http://localhost` -> works locally but not remotely = network/firewall. Fails locally = Apache/app config.
4. Test by external IP from outside, then by domain name -> works by IP but not name = **DNS**.
5. Firewalls: `firewall-cmd --list-all` / `ufw status`, then cloud Security Group/NACL/load balancer target health.
6. **SELinux** (RHEL): `getenforce`; wrong file context on a custom DocumentRoot (`restorecon -Rv`), or `httpd_can_network_connect` off when proxying to a backend.
7. vhost/DocumentRoot: correct `ServerName`, `<Directory>` has `Require all granted`, files readable and parent directories have `x` for the apache user.
8. Logs tell the story: `error_log` (403 permission denied, AH00035) and `access_log` (are requests even arriving?).
9. HTTPS only failing: certificate path, expired cert, port 443 not open.

Example:

```
sudo apachectl configtest              # Syntax OK
sudo ss -tlnp | grep httpd
curl -I http://localhost
sudo firewall-cmd --add-service={http,https} --permanent && sudo firewall-cmd --reload
sudo tail -f /var/log/httpd/error_log
getenforce; sudo restorecon -Rv /var/www/mysite
sudo setsebool -P httpd_can_network_connect on
```

What to say to the interviewer: *I verify the config with* `apachectl configtest`*, confirm with* `ss` *that it listens on the right interface and port, then compare curl on localhost with curl from outside to split an Apache problem from a network one. After that I check firewalld or ufw, security groups, SELinux contexts and booleans, vhost and DocumentRoot permissions, DNS, and the error and access logs.*

Q71. Automate an application deployment with a shell script, with backup, health check and automatic rollback.

In simple words: Like changing a shop's display overnight: keep the old display packed next to you, set up the new one, switch the lights on and check it looks right - if not, put the old one back before customers arrive.

Detailed answer

Design used by real tools (Capistrano-style):

- Each release goes into its own timestamped folder: `/opt/app/releases/20260924_101500`.
- A symlink `/opt/app/current` points to the live release -> switching is **atomic** (`ln -sfn`), and rollback is just pointing back.
- Keep the last N releases for quick rollback; delete older ones.
- Health check in a retry loop after restart; on failure, switch the symlink back and restart.
- `set -euo pipefail` + `trap` so a failed step never leaves things half-done silently.

Example:

```
#!/usr/bin/env bash
# Usage: ./deploy.sh /tmp/app-build.tar.gz
set -euo pipefail
ARTIFACT="${1:?Usage: $0 <artifact.tar.gz>}"
APP=/opt/app; SERVICE=myapp; HEALTH=http://localhost:8080/health; KEEP=5
NEW="$APP/releases/$(date +%Y%m%d_%H%M%S)"
PREV="$(readlink -f "$APP/current" || true)"

log() { echo "$(date '+%F %T') $*"; }
rollback() {
  log "Deployment FAILED - rolling back to $PREV"
  [ -n "$PREV" ] && ln -sfn "$PREV" "$APP/current" && systemctl restart "$SERVICE"
  exit 1
}
trap rollback ERR

[ -f "$ARTIFACT" ] || { log "Artifact not found"; exit 2; }
mkdir -p "$NEW" && tar -xzf "$ARTIFACT" -C "$NEW"
log "Switching current -> $NEW"
ln -sfn "$NEW" "$APP/current"
systemctl restart "$SERVICE"

for i in $(seq 1 10); do
  if curl -fsS --max-time 3 "$HEALTH" >/dev/null; then
    log "Healthy after ${i} checks"; trap - ERR
    ls -1dt "$APP"/releases/* | tail -n +$((KEEP+1)) | xargs -r rm -rf
    exit 0
  fi
  sleep 3
done
false     # health never passed -> triggers ERR trap -> rollback
```

What to say to the interviewer: *My script extracts each build into a timestamped release directory, atomically switches a* `current` *symlink, restarts the service with systemctl and polls the health endpoint in a loop. An ERR trap under* `set -euo pipefail` *points the symlink back to the previous release and restarts it if any step or the health check fails, and old releases are pruned to keep the last five.*

Q72. An application fails intermittently. How do you use Linux command-line tools to analyse its logs?

In simple words: Intermittent failures are like a car that stalls "sometimes". You go through the trip log, count how often and at what times it happened, and look at what else happened at the same minute.

Detailed answer

1. Narrow the time window: `journalctl -u myapp --since "2026-09-24 10:00" --until "10:30"`.
2. Find errors with context: `grep -E "ERROR|Exception|timeout" -C 5 app.log` (lines before/after show the cause).
3. Count and group to find patterns:

- `grep -c` for totals; `sort | uniq -c | sort -rn` for the most common messages.
- `awk` to count HTTP status codes per minute from access logs -> shows spikes.

4. Include rotated logs: `zgrep` on `.gz` files.

5. Watch live while reproducing: `tail -F` (follows through rotation) or `less +F`.
6. **Correlate timestamps** with other sources: `dmesg -T` (OOM kills, disk errors), system journal, cron times, deploy times, database logs.
7. Check if failures cluster on one host, one endpoint or one upstream.

Example:

```
journalctl -u myapp --since "1 hour ago" -p err
grep -E "ERROR|Timeout" -C 3 /var/log/myapp/app.log | less
# Top error messages
grep ERROR app.log | sed 's/^[^]]*] //' | sort | uniq -c | sort -rn | head
# 5xx per minute from nginx access log ($9 = status, $4 = [time)
awk '$9 ~ /^5/ {print substr($4,2,17)}' /var/log/nginx/access.log | uniq -c | sort -rn | head
zgrep -h "Timeout" /var/log/myapp/app.log*.gz | wc -l
dmesg -T | grep -i "killed process"
tail -F /var/log/myapp/app.log | grep --line-buffered ERROR
```

What to say to the interviewer: *I narrow the time window with* `journalctl --since`*, grep errors with context lines, and use* `sort | uniq -c` *and awk to count error types and 5xx per minute, which reveals patterns. I include rotated logs with zgrep, follow live with* `tail -F` *while reproducing, and correlate the timestamps with dmesg OOM events, cron and deployments.*

Q73. A critical service fails to start after a reboot. How do you troubleshoot it?

In simple words: After a power cut, one machine will not start. Maybe it was never set to start automatically, it tried to start before the electricity or water it needs was ready, or someone left something blocking it.

Detailed answer

1. Was it enabled? `systemctl is-enabled myapp` -> if `disabled`, it never tried: `systemctl enable myapp`.
2. What failed? `systemctl --failed`, `systemctl status myapp`, and `journalctl -b -u myapp` (this boot only).
3. **Ordering/dependencies**: it started before what it needs. Network apps should use `After=network-online.target` + `Wants=network-online.target`; add `After=`/`Requires=` for databases or mounts (`RequiresMountsFor=/data`).
4. **Mounts not ready**: a data disk or NFS missing from fstab, or it failed -> app cannot find its files. Use `nofail`/`_netdev` so a bad mount does not break boot.
5. **Port already in use**: another service grabbed it (`ss -tlnp`).
6. Permissions / env files: `EnvironmentFile=` missing, `/run` or `/tmp` directories not recreated (use `RuntimeDirectory=`).
7. SELinux denials: `ausearch -m avc -ts boot`.
8. Unit edited but not reloaded -> `systemctl daemon-reload`.
9. Add `Restart=on-failure` so a transient issue heals itself.

Example:

```
systemctl is-enabled myapp
systemctl --failed
journalctl -b -u myapp --no-pager | tail -30
systemctl list-dependencies myapp
sudo systemctl edit myapp
# [Unit]
# After=network-online.target postgresql.service
# Wants=network-online.target
# RequiresMountsFor=/data
sudo systemctl daemon-reload && sudo systemctl restart myapp
```

What to say to the interviewer: *I check whether the unit is enabled, then read* `journalctl -b -u` *and* `systemctl --failed` *for the actual error. The common causes after a reboot are ordering problems, fixed with* `After=network-online.target` *or* `RequiresMountsFor`*, a mount that did not come up, a port already taken, missing environment or runtime files, or SELinux, and I add* `Restart=on-failure` *for resilience.*

Q74. How would you tune the kernel with sysctl for a high-traffic application?

In simple words: The kernel ships with settings for an average house. A busy stadium needs wider gates and a longer waiting area - but you only widen the gates that are actually crowded, and you measure before and after.

Detailed answer

Common knobs for many concurrent connections:

Setting	What it controls	Typical high-traffic value
`net.core.somaxconn`	max accept queue per listening socket	4096-65535 (app backlog must match)
`net.ipv4.tcp_max_syn_backlog`	half-open connection (SYN) queue	8192+
`net.core.netdev_max_backlog`	packets queued from the NIC	16384+
`net.ipv4.ip_local_port_range`	ports for OUTGOING connections (proxies)	`1024 65535`
`net.ipv4.tcp_tw_reuse`	reuse TIME_WAIT sockets for outgoing	1
`fs.file-max` + `LimitNOFILE`	total and per-process open files/sockets	e.g. 2M and 65536
`vm.swappiness`	how eagerly to swap	1-10 for latency-sensitive apps

Rules:

- **Measure first**: `ss -s`, `netstat -s | grep -i -E "overflow|drop"` (listen queue overflows), TIME_WAIT counts, fd usage. Tune only what the data shows is a bottleneck.
- Persist in `/etc/sysctl.d/99-app.conf`, apply with `sysctl --system`; change one thing at a time; load test and compare.
- Avoid cargo-cult tuning copied from blogs: `tcp_tw_recycle` was removed from Linux (it broke clients behind NAT), and huge buffers can increase latency.
- In Kubernetes, many of these are namespaced and set per pod (`securityContext.sysctls`), others only on the node.

Example:

```
ss -s
netstat -s | grep -i "listen"          # "times the listen queue of a socket overflowed"
cat <<'EOF' | sudo tee /etc/sysctl.d/99-webapp.conf
net.core.somaxconn = 8192
net.ipv4.tcp_max_syn_backlog = 8192
net.core.netdev_max_backlog = 16384
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
vm.swappiness = 10
EOF
sudo sysctl --system
```

> ***What to say to the interviewer:*** *I first measure with* `ss -s` *and* `netstat -s` *for listen queue overflows, TIME_WAIT and file descriptor usage, and only then tune the relevant values such as somaxconn, tcp_max_syn_backlog, ip_local_port_range, tcp_tw_reuse and file limits. I persist them in /etc/sysctl.d, change one thing at a time with load tests before and after, and avoid copying blog settings like the removed tcp_tw_recycle.*

Q75. How do you deploy updates with zero downtime on a plain Linux server (no Kubernetes)?

> **In simple words:** Changing the cashier at a checkout without closing it: the new cashier sits down at the next counter, the manager starts sending new customers there, and the old cashier finishes serving the people already in line before leaving.

Detailed answer

Techniques (often combined):

1. **Two or more instances behind a reverse proxy** (nginx/HAProxy) or a load balancer: take instance A out of rotation (drain), update and health-check it, put it back, then do B - a **rolling restart**.
2. **Connection draining**: let in-flight requests finish before stopping (LB deregistration delay, HAProxy `drain` state, app handling SIGTERM gracefully).
3. **Graceful reload of the proxy**: `nginx -t && nginx -s reload` (or `systemctl reload nginx`) starts new workers with the new config while old workers finish their requests - no dropped connections.
4. **Atomic symlink switch + graceful app reload**: deploy to a new release folder, switch `current` symlink, then graceful reload (many app servers like gunicorn, php-fpm or puma support reloading workers on a signal without closing the listening socket).
5. **Blue-green on one host**: run the new version on another port, health-check it, flip the nginx upstream, reload nginx, keep the old one for instant rollback.
6. **systemd socket activation**: systemd holds the listening socket, so connections queue briefly during a restart instead of being refused.
7. Database changes must be backward compatible (expand/contract) so old and new versions can run together.

Example:

```
upstream app {
    server 127.0.0.1:8081;       # blue (current)
    server 127.0.0.1:8082 down;  # green (new) - flip 'down' after health check
}
```

```
curl -fsS http://127.0.0.1:8082/health && \
  sudo sed -i 's/8081;/8081 down;/; s/8082 down;/8082;/' /etc/nginx/conf.d/app.conf && \
  sudo nginx -t && sudo nginx -s reload
```

> ***What to say to the interviewer:*** *I run at least two app instances behind nginx or HAProxy and do a rolling update: drain one, deploy and health-check it, return it to the pool, then the next, with the proxy itself changed only by graceful reloads. On a single host I use blue-green on two ports or an atomic symlink switch plus the app server's graceful reload, and I keep database changes backward compatible so both versions can run together.*

Q76. Design a 3-tier architecture (web, app, DB), explain the request flow, and debug a failure layer by layer with Linux commands.

> **In simple words:** A restaurant: the waiter at the front (web tier) takes the order, the kitchen (app tier) cooks it, and the storeroom (database) holds the ingredients. When a customer complains, you walk the same path the order took and check each station in turn, instead of guessing.

Detailed answer

Design:

```
 User --DNS--> app.example.com
   |
   v   HTTPS 443
[ Load balancer / nginx (web tier) ]   public subnet, TLS termination, static files
   |            |            (health checks, least_conn)
   v HTTP 8080  v
[ App server 1 ]  [ App server 2 ]     private subnet, stateless, sessions in Redis
   |            |
   v TCP 5432   v
[ DB primary ] ---replication---> [ DB replica ]   private DB subnet, Multi-AZ
```

- Each tier sits in its own subnet with security groups allowing only the tier above it (LB -> app 8080, app -> DB 5432).
- Web tier scales horizontally, app tier is stateless behind the LB, DB has a primary plus a replica for failover and reads.
- Flow: DNS resolves -> TCP + TLS to the LB -> LB proxies to a healthy app server -> app queries the DB -> response travels back the same way.

Debug, layer by layer (what to run and why):

1. **Scope first** - all users or some, all pages or one endpoint, since when, what changed (deploy, config, certificate)?
2. **DNS** - `dig +short app.example.com` -> does the name resolve, to the right IP (stale record, wrong TTL)?
3. **Connectivity** - `nc -zv <lb-ip> 443` and `curl -v https://app.example.com/health` -> is the port reachable, and what status comes back (timeout = network/SG; 502/504 = backend problem)?

4. **TLS** - `openssl s_client -connect app.example.com:443 -servername app.example.com` -> expired certificate, broken chain, wrong hostname?
5. **Web tier** - `nginx -t` (config valid?), `ss -tlnp` (listening on 443?), `tail -f /var/log/nginx/error.log` ("upstream timed out" or "connection refused" tells you the app tier is the problem), `access.log` for status codes and `$upstream_response_time`.
6. **App tier** - `systemctl status myapp` and `journalctl -u myapp -f` (crashing, exceptions?), `curl localhost:8080/health` on the box (bypasses the LB), `top` (CPU/memory saturated?), `lsof -p <PID> | wc -l` (fd leak), `strace -p <PID>` (stuck waiting on a socket = waiting on the DB?), `ss -tnp | grep 5432` (connection pool full?).
7. **DB tier** - `nc -zv db 5432`; connections vs `max_connections` (`SELECT count(*) FROM pg_stat_activity;`), long or blocked queries (`pg_stat_activity`, slow query log), locks, replication lag; on the host `iostat -x 1` (disk saturated?), `df -h` (disk full stops writes).
8. **Network in between** - `mtr` (loss or latency per hop), `ss -s` and `netstat -s | grep -i retrans` (retransmits), `tcpdump -i eth0 port 5432` (are packets leaving and replies coming back?), MTU (`ping -M do -s 1472 <ip>` - big packets hang while small ones work), security groups/NACLs.
9. **Fix and verify** - roll back the change or scale/restart the failing tier, confirm with the same checks and dashboards, then write the RCA.

Example:

```
dig +short app.example.com
curl -sv -o /dev/null -w "%{http_code} %{time_total}\n" https://app.example.com/health
openssl s_client -connect app.example.com:443 -servername app.example.com </dev/null 2>/dev/null \
  | openssl x509 -noout -dates
sudo nginx -t && sudo tail -20 /var/log/nginx/error.log
# upstream timed out (110) while reading response header from upstream: 10.0.2.11:8080
ssh app1 'curl -s localhost:8080/health; journalctl -u myapp -n 30 --no-pager'
ssh app1 'ss -tnp | grep -c ":5432"'               # 100 = pool exhausted?
psql -h db -c "SELECT state, count(*) FROM pg_stat_activity GROUP BY state;"
mtr -rwc 30 10.0.3.10; netstat -s | grep -i retrans
```

> ***What to say to the interviewer:*** *I design it as a load balancer or nginx web tier in a public subnet, stateless app servers in a private subnet, and a primary database with a replica in a DB subnet, each tier only reachable from the one above. To debug I follow the request path: DNS with dig, reachability and status with curl and nc, TLS with openssl, then the nginx error log to see whether upstream failed, the app's journal, resources and strace, the database's connections, slow queries and disk, and finally mtr, retransmits, tcpdump and MTU for the network in between.*

Q77. You are locked out of a server over SSH and have no root access. How do you recover?

> **In simple words:** You lost the key to your flat. Instead of breaking the door, you use the building manager's master access (the cloud console), or you take the lock out, fix it on a workbench, and put it back.

Detailed answer

On AWS / cloud (in order of convenience):

1. **SSM Session Manager** -> if the SSM agent and instance role are there, you get a shell with no SSH at all: `aws ssm start-session --target i-0abc`.
2. **EC2 Instance Connect** or the **EC2 Serial Console** (Nitro instances) -> console access even when networking or sshd is broken.
3. **Rescue-volume method:** stop the instance -> detach the root EBS volume -> attach it to a helper instance in the same AZ -> mount it -> fix `home/ec2-user/.ssh/authorized_keys`, `etc/ssh/sshd_config` or `etc/sudoers.d/` -> unmount -> reattach as the root device -> start.
4. **User data:** on some AMIs, cloud-init user data set to run on every boot can re-inject a public key.

On-prem / VM: use the hypervisor or iLO/iDRAC console, boot to single-user mode (`systemd.unit=rescue.target`, or `rd.break` on RHEL to reset the root password), then fix the problem.

Afterwards: find out why it happened (a bad sshd_config, full disk, changed key, fail2ban ban, SG change), and enable SSM so it never needs this again.

Example:

```
# Rescue method on a helper instance (root volume attached as /dev/xvdf)
sudo lsblk
sudo mkdir -p /mnt/rescue && sudo mount /dev/xvdf1 /mnt/rescue
cat ~/new_key.pub | sudo tee -a /mnt/rescue/home/ec2-user/.ssh/authorized_keys
sudo chroot /mnt/rescue sshd -t -f /etc/ssh/sshd_config   # validate config
sudo umount /mnt/rescue
```

> ***What to say to the interviewer:*** *On AWS I first try SSM Session Manager or the EC2 serial console, which don't depend on SSH. If neither is available, I stop the instance, attach its root volume to a helper instance, fix authorized_keys, sshd_config or sudoers, and reattach it. On-prem I use the out-of-band console and rescue mode. Then I fix the root cause and make sure SSM is enabled everywhere.*

Q78. Write a shell script that checks whether a service is running, restarts it if not, and logs the event.

> **In simple words:** A night watchman who checks that the lights are on. If they're off, he flips the switch, writes it in the logbook, and calls someone if the switch won't work.

Detailed answer

- Use `systemctl is-active --quiet` to check each service.
- Restart, then **verify** that the restart actually worked; don't assume it did.
- Log to syslog/journald with `logger`, so logs are centralized and rotated.
- Return a non-zero exit code if anything is still down, so cron or monitoring can alert.
- Note: `Restart=on-failure` in the systemd unit usually makes this script unnecessary. Use the script for extra checks or alerting.

Example:

```
#!/usr/bin/env bash
# svc_guard.sh - restart services that are down, log it, and report failures
set -uo pipefail
SERVICES=("${@:-nginx}")
failed=0

for svc in "${SERVICES[@]}"; do
  if systemctl is-active --quiet "$svc"; then
    continue
  fi
  logger -t svc_guard -p user.warning "$svc is down - restarting"
  if systemctl restart "$svc" && sleep 3 && systemctl is-active --quiet "$svc"; then
    logger -t svc_guard "$svc restarted successfully"
  else
    logger -t svc_guard -p user.err "$svc FAILED to restart: $(systemctl is-failed "$svc")"
    failed=1
  fi
done
exit "$failed"
```

```
# run every 2 minutes; view the history with: journalctl -t svc_guard
*/2 * * * * root /opt/scripts/svc_guard.sh nginx docker
```

```
# Usually the better fix - let systemd do it:
# /etc/systemd/system/nginx.service.d/restart.conf
[Service]
Restart=on-failure
RestartSec=5
```

> ***What to say to the interviewer:*** *The script loops over the services, uses* `systemctl is-active` *to check each one, restarts any that are down, verifies the restart worked, logs everything with logger into journald, and exits non-zero if a service is still down so monitoring alerts. I'd point out that* `Restart=on-failure` *in the systemd unit handles most of this natively, so the script is mainly for alerting.*

Q79. Networking: a remote user cannot connect to the company VPN. How do you troubleshoot?

> **In simple words:** Check the problem from the user's side outward: is their internet working, can they reach the VPN gate, does the gate recognise their ID, and once inside, do they get a map (routes and DNS)?

Detailed answer

1. **Scope:** is it only this user or everyone? If everyone, look at the VPN gateway itself (service down, certificate expired, licence limit).
2. **User's internet:** can they browse? Hotel or corporate Wi-Fi often blocks UDP 500/4500 (IPsec) or 1194 (OpenVPN), so try a TCP/443 profile.
3. **Reachability:** `nslookup vpn.company.com`, `nc -vz vpn.company.com 443`, or check the UDP ports.
4. **Authentication:** wrong password, expired account or certificate, MFA problem, clock skew (certificate and TOTP validity). Check the VPN server and IdP logs for the user.

5. **Connected but nothing works:** no route pushed (split tunnel), overlapping subnets (the home LAN is 192.168.1.0/24, the same as the office), DNS not pushed, MTU issues (large packets drop, so lower the MTU or set MSS clamping).
6. **AWS Client VPN / Site-to-Site:** authorization rules, route table entries for the target subnet, the SG on the endpoint's association, tunnel status in CloudWatch.

Example:

```
nslookup vpn.company.com
nc -vz vpn.company.com 443
# after connecting:
ip route | grep tun0          # were routes pushed?
ping -M do -s 1372 10.10.0.10 # MTU test; lower the size if it fails
resolvectl status | grep -A2 tun0
```

> ***What to say to the interviewer:*** *I first check whether it's one user or everyone. Then I check the user's own internet and whether VPN ports are blocked on their network, then authentication (certificate expiry, MFA, clock skew) in the server logs. If they connect but can't reach anything, I look at the pushed routes, overlapping home and office subnets, DNS and MTU. On AWS Client VPN I also check the authorization rules and route table.*

Q80. Networking: users report slow access between two branch offices over a WAN link (or a remote office cannot reach the main server). How do you troubleshoot?

Also asked as: A user in a remote office cannot access the company's main server. How would you diagnose it?

> **In simple words:** Traffic between offices travels on a motorway you share with everyone. It can be slow because the road is full (bandwidth), too long (latency), full of potholes (packet loss), or the trucks are too tall for the bridges (MTU).

Detailed answer

1. **Scope:** one user, one app, or the whole office? All the time or only at peak hours?
2. **Can't reach at all:** test step by step: local gateway -> WAN router -> remote side (`traceroute`/`mtr`). Check that the VPN/MPLS tunnel is up, the routes for the server subnet exist on both ends, firewall/ACL/SG rules, and DNS (does the name resolve to the internal IP?).
3. **Slow:** measure latency and loss with `mtr` over several minutes. Measure throughput with `iperf3` against link capacity, and look at interface utilization and errors (SNMP/NetFlow).
4. **Common causes:** a saturated link (backups or updates at the wrong time), no QoS for critical traffic, packet loss on the ISP, MTU/fragmentation over the tunnel, a chatty application (SMB/database) that suffers from high round-trip time, duplex mismatch or interface errors.
5. **Fixes:** QoS, schedule bulk transfers off-peak, increase bandwidth or add a second link, WAN optimisation or caching, MSS clamping on the tunnel, and move chatty apps closer to users. In the cloud, consider Direct Connect instead of internet VPN.

Example:

```
mtr -rwc 100 10.20.0.15           # latency and loss per hop over 100 probes
iperf3 -c 10.20.0.15 -t 30        # throughput (iperf3 -s running on the far side)
ping -M do -s 1400 10.20.0.15     # MTU through the tunnel
ip -s link show eth0              # errors and drops on the interface
```

What to say to the interviewer: *I scope it first: which users, which apps, and when. If it's unreachable, I trace hop by hop and check the tunnel, routes on both sides, firewall rules and DNS. If it's slow, I use mtr for latency and loss, iperf3 for throughput against the link's capacity, and interface counters for errors. The usual culprits are a saturated link, no QoS, ISP packet loss or MTU problems over the tunnel.*

Q81. Networking: the network has intermittent connectivity issues. What could cause it, and how do you troubleshoot?

In simple words: A light that flickers is harder to fix than one that's off, because you have to catch it in the act. So you set up continuous recording and look for a pattern in time, place or load.

Detailed answer

Likely sources:

- Physical: a bad cable or SFP, a flapping interface, Wi-Fi interference, a duplex mismatch.
- Capacity: link saturation at peaks, NAT port exhaustion (e.g. NAT Gateway `ErrorPortAllocation`), conntrack table full (`nf_conntrack: table full` in dmesg).
- Addressing: duplicate IP / ARP conflicts, DHCP lease problems.
- Routing: a route flapping between two paths, BGP session resets, asymmetric routing through a stateful firewall.
- DNS: one of several resolvers failing, so only some lookups fail.
- Cloud: instance bandwidth or PPS limits, health checks toggling targets, spot interruptions.

Approach:

1. Get timestamps and a pattern: which hosts, when, and which destinations.
2. Run continuous tests (`mtr`, `ping -D` with timestamps, synthetic checks) and correlate with metrics: interface errors, drops, CPU on network devices, NAT/conntrack counters.
3. Capture during an event (`tcpdump` on both ends) to see retransmits, resets or ARP storms.
4. Change one thing at a time and confirm the pattern disappears.

Example:

```
ping -D -i 0.5 10.0.1.20 | tee ping.log         # timestamped, catch the drop window
ip -s link show eth0                            # rx/tx errors, dropped
dmesg -T | grep -Ei 'conntrack|link is down|duplex'
arping -D -I eth0 10.0.1.20                     # duplicate IP detection
ethtool -S eth0 | grep -i allowance             # AWS ENA bandwidth/PPS limits hit
```

What to say to the interviewer: *Intermittent problems need data over time, so I collect timestamps and run continuous pings or mtr to find a pattern. Then I correlate with interface errors, NAT and conntrack exhaustion, ARP or duplicate-IP conflicts, route flaps, DNS resolvers and cloud bandwidth limits, and capture packets during an event. Once I have a hypothesis, I change one thing at a time and confirm the drops stop.*

Q82. Networking: traffic is not being forwarded between two subnets. How do you troubleshoot the routing?

In simple words: Two neighbourhoods and one junction between them. Each house must know "for the other neighbourhood, go via the junction", the junction must be willing to pass traffic, and the reply must be able to come back the same way.

Detailed answer

1. **Hosts:** is the default gateway correct on both sides (`ip route`)? Is the subnet mask right? A wrong mask makes a host think the other subnet is local.
2. **Router or Linux box:** does the routing table have both subnets (connected or static)? On Linux, is IP forwarding on (`sysctl net.ipv4.ip_forward=1`)?
3. **Return path:** traffic may go out fine but the reply has no route back. Asymmetric routing through a firewall also drops it.
4. **Filters:** ACLs on router interfaces, the host firewall, and in AWS the NACLs (stateless, so both directions and the ephemeral ports 1024-65535 must be allowed) and Security Groups.
5. **AWS specifics:** subnets in the same VPC route via the `local` route automatically, so if they can't talk it's the SG or NACL. Between VPCs, check the peering/TGW routes in BOTH route tables, TGW route table associations, and that the CIDRs don't overlap. For an EC2 acting as a router or NAT, disable source/destination check.
6. **Verify:** `traceroute` shows where packets stop; `tcpdump` on the router shows whether packets arrive and leave.

Example:

```
ip route get 10.0.2.25                # which route/interface would be used
sysctl net.ipv4.ip_forward            # must be 1 on a Linux router
sudo tcpdump -ni eth1 host 10.0.2.25  # does traffic leave the router?
aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=subnet-0abc
```

What to say to the interviewer: *I check the hosts' gateway and subnet mask, then the router's routing table and IP forwarding, then the return path, because routing failures are often one-way. Then I check ACLs and firewalls. In AWS, subnets in one VPC always have a local route, so it's usually NACLs or Security Groups, while between VPCs I check both route tables and CIDR overlap. traceroute and tcpdump show exactly where packets stop.*

Q83. Networking: you need to add a new subnet to an existing network. What are the steps?

In simple words: Adding a new street to a town: pick house numbers that nobody else is using, connect the street to the road map, and put up the right gates.

Detailed answer

1. **Plan the CIDR:** pick a free range inside the VPC or network that doesn't overlap existing subnets, on-prem ranges, peered VPCs or VPN clients. Size it for growth: a /24 gives 256 addresses, and AWS reserves 5 per subnet, so 251 are usable. EKS pods using VPC CNI need large subnets.

2. **Subnetting math:** 10.0.0.0/16 split into /20s gives 16 subnets of 4096 addresses: 10.0.0.0/20, 10.0.16.0/20, 10.0.32.0/20, and so on.
3. **Create it** in the right AZ (AWS) or VLAN/interface (on-prem). A VLAN is layer 2 separation; a subnet is the layer 3 range you put on it.
4. **Routing:** associate the right route table: public (0.0.0.0/0 -> IGW) or private (0.0.0.0/0 -> NAT). Add routes on peering, TGW, VPN or on-prem routers so other networks can reach it.
5. **Security:** NACLs and SGs, and update firewall rules and allow-lists that reference CIDRs.
6. **Services:** DHCP scope or auto-assign public IP setting, DNS, VPC endpoints, tags. Then do it all in Terraform, and test connectivity from both directions.

Example:

```
# Quick subnet math
ipcalc 10.0.48.0/20        # Network, Broadcast, HostMin/HostMax, Hosts/Net: 4094

aws ec2 create-subnet --vpc-id vpc-0abc --cidr-block 10.0.48.0/20 \
  --availability-zone ap-south-1c \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=private-1c}]'
aws ec2 associate-route-table --route-table-id rtb-0priv --subnet-id subnet-0new
```

> ***What to say to the interviewer:*** *I pick a non-overlapping CIDR sized for growth, remembering AWS reserves five addresses per subnet. I create the subnet in the right AZ, associate the public or private route table, and add routes on peering, TGW or VPN so other networks can reach it. Then I update NACLs, Security Groups and allow-lists, do it all in Terraform, and test from both directions.*

Q84. Networking: you notice abnormal traffic patterns on an interface. How do you investigate and what do you recommend?

In simple words: The water meter suddenly spins much faster than usual. You find out which tap is running, whether it's a guest having a long shower (legitimate spike) or a burst pipe (attack or leak).

Detailed answer

1. **Quantify:** the baseline vs now, inbound vs outbound, bytes vs packets (a high PPS with small packets suggests a DDoS or scan).
2. **Top talkers:** which source/destination IPs and ports? Use NetFlow/sFlow, **VPC Flow Logs** (query with Athena or CloudWatch Logs Insights), `iftop` or `nethogs` on the host.
3. **Classify:** legitimate (a deploy, backup, new customer, crawler), misconfiguration (a retry storm, a logging loop, traffic crossing AZs or going through NAT), or malicious (DDoS, port scans, crypto-mining, data exfiltration to unknown IPs).
4. **Deep dive:** `tcpdump` a sample, check GuardDuty findings, and check the process on the host that opened those connections.
5. **Respond:** block at the right layer (NACL/WAF rate rule/Shield for DDoS, SG for a host), isolate a compromised host, rotate its credentials, and fix the misconfiguration.
6. **Recommend:** baselines and anomaly alerts on flow metrics, egress filtering, WAF rate limiting, and GuardDuty/IDS.

Example:

```
-- Athena on VPC Flow Logs: top talkers in the last hour
SELECT srcaddr, dstaddr, dstport, SUM(bytes)/1048576 AS mb
FROM vpc_flow_logs
WHERE start > to_unixtime(now() - interval '1' hour)
GROUP BY srcaddr, dstaddr, dstport
ORDER BY mb DESC LIMIT 20;
```

```
sudo iftop -i eth0 -P           # live per-connection bandwidth
sudo nethogs eth0               # which process is using it
sudo tcpdump -ni eth0 -c 200 'not port 22'
```

> ***What to say to the interviewer:*** *I compare against the baseline, then find the top talkers with flow logs or NetFlow and iftop on the host, and classify it: a legitimate spike, a misconfiguration like a retry loop, or malicious activity like a DDoS or exfiltration. I confirm with tcpdump and GuardDuty, block or isolate at the right layer, and recommend baselines with anomaly alerts, egress filtering and WAF rate limits.*

Q85. Networking: how do you implement redundancy for critical network devices and links?

> **In simple words:** Never have only one bridge into town. Build two, on different routes, and have a traffic officer who instantly sends cars to the second bridge if the first one closes.

Detailed answer

Options (on-prem):

- **Redundant devices:** two routers/firewalls sharing a virtual gateway IP with **VRRP/HSRP** (active/standby), or firewall HA pairs.
- **Redundant links:** dual ISPs, link aggregation (LACP) for switch uplinks, and diverse physical paths.
- **Dynamic routing** (BGP/OSPF) so traffic reroutes automatically when a path fails.
- **Power and hardware:** dual PSUs, separate power feeds, spares on site.

Options (cloud/hybrid):

- Spread across **multiple AZs**: NAT Gateway per AZ, load balancers across AZs, subnets in each AZ.
- **Site-to-Site VPN** has two tunnels per connection, so configure both. Use two customer gateways for device redundancy.
- **Direct Connect** with a second DX at a different location, or DX + VPN as backup, with BGP deciding the path.
- Route 53 health-check failover for public endpoints.

Recommendation: active/standby pairs with VRRP for gateways, two diverse links with BGP failover, and in AWS multi-AZ everything plus both VPN tunnels. Test failover regularly, because untested redundancy often fails when you need it.

Example:

```
           ISP-A ----\                      /---- AZ-a: NAT GW, app, DB primary
 Office  [FW1]==VRRP==[FW2] ==BGP== AWS TGW ==
           ISP-B ----/  (2 VPN tunnels + DX)  \---- AZ-b: NAT GW, app, DB standby
```

What to say to the interviewer: *I remove single points of failure at each layer: device pairs with VRRP or HSRP, two diverse links or ISPs with BGP for automatic failover, and dual power. In AWS that means multi-AZ subnets and a NAT gateway per AZ, both VPN tunnels up, and Direct Connect backed by a VPN. Most importantly I test failover regularly.*

Q86. Networking: a router is not learning routes from its neighbours (BGP/OSPF). How do you troubleshoot?

In simple words: Two neighbours only share gossip (routes) once they have properly introduced themselves. First check that they're actually talking, then that they agreed on the rules, then that nobody is filtering what's being said.

Detailed answer

1. **Is the neighbour relationship up?** BGP: the state should be `Established`; if it's stuck in `Idle`/`Active`/`Connect`, it's a connectivity or config issue. OSPF: the state should be `Full`.
2. **Connectivity:** can the routers ping each other on the peering IPs? Is TCP 179 (BGP) allowed through ACLs and firewalls? Is OSPF (IP protocol 89) or multicast allowed?
3. **Parameters must match:** BGP neighbour IP and **ASN**, MD5 password, eBGP multihop/TTL. OSPF area ID, hello/dead timers, subnet mask, authentication, MTU (mismatched MTU gets stuck in ExStart), network type.
4. **Established but no routes:** inbound/outbound **filters** (prefix-lists, route-maps), prefixes not advertised (a missing `network` statement or redistribution), max-prefix limit hit, next-hop unreachable, or a better route already in the table (administrative distance).
5. **AWS angle:** for Site-to-Site VPN and Direct Connect, check the tunnel/BGP status in the console or CloudWatch (`TunnelState`), the ASN on the customer gateway vs the VGW/TGW, the advertised prefixes (limit of 100 for VPN), and route propagation enabled on the VPC route tables.

Example:

```
router# show ip bgp summary          # neighbour state, prefixes received
router# show ip bgp neighbors 169.254.10.1 advertised-routes
router# show ip ospf neighbor        # FULL / EXSTART / INIT
router# show ip route bgp
```

```
aws ec2 describe-vpn-connections --query 'VpnConnections[].VgwTelemetry'
aws ec2 describe-route-tables --query 'RouteTables[].PropagatingVgws'
```

What to say to the interviewer: *First I check whether the adjacency is up: BGP Established or OSPF Full. If not, I check connectivity, TCP 179 or OSPF traffic through firewalls, and matching parameters like ASN, passwords, areas, timers and MTU. If the session is up but no routes arrive, it's usually filters, prefixes not advertised, or max-prefix limits. For AWS VPN or Direct Connect, I also check the tunnel telemetry, the ASNs and route propagation on the VPC route tables.*